

1 绪论

1.1 嵌入式文件系统特点

随着社会的信息化和网络化的日益加强，对计算机系统小型化、实时性、可靠性的要求越来越高，嵌入式系统正是在这种背景下应运而生。目前嵌入式系统的应用日益广泛，大到波音飞机小到 MP3、移动电话都有嵌入式系统的存在。嵌入式系统是以应用为中心、以计算机技术为基础、软件硬件可裁剪、功能、可靠性、成本、体积、功耗严格要求的专用计算机系统。所谓文件系统，是操作系统中藉以组织、存储和命名文件的结构。

文件系统方便了对存储空间的管理：1）统一管理存储空间，实施存储空间的分配与回收；2）实现文件的按名存取，使用方便、直观；3）向用户提供一个方便使用的接口，提供对文件系统操作命令，以及提供对文件的操作命令；4）优化存储速度和空间，屏蔽物理因素的影响，如对于坏损单元采用透明的坏损管理。

嵌入式系统使用的存储器一般为 Flash，基于 Flash 芯片本身的特性和嵌入式系统的应用环境，一般对嵌入式 Flash 文件系统有如下的要求：

（1）掉电安全机制(crash/powerdown-safe)。由于嵌入式系统的运行环境一般都比较恶劣，但同时又要求系统有较高的可靠性，即在系统突然掉电或者重新启动后，不能破坏 Flash 中的数据，同时保持文件系统的一致性和完整性。

（2）均衡磨损 (wear_leveling)。由于 Flash 的擦除次数有限制，所以文件系统要均衡使用 Flash 的每个扇区，延长 Flash 的使用寿命。

（3）垃圾回收机制(garbage collection)。在存储器分配使用一段时间后，Flash 中可能存在脏数据，这就要进行垃圾回收，将无效的数据从存储器中清除来保证高效地使用存储器。因为 Flash 的擦除以扇区为单位，所以垃圾回收也应该以扇区为单位进行，将有效的数据全部移动到另一个扇区后再擦除整个扇区。

（4）坏块管理(dirty trick management)。由于 NAND Flash 的工艺不能保证 NAND Flash 的 Memory Array 在其生命周期中保持性能的可靠，因此，在 NAND Flash 的生产中及使用过程中会产生坏块。文件系统要能识别并标记其中的坏块，实现对文件系统函数的透明调用。

1.2 Flash 存储器

Flash 存储器是一种静态存储设备，其所有的部件都是固定的。Flash 存储器按其底层技术结构主要分为 NAND Flash 和 NOR Flash 两大类。所谓 NOR Flash 和 NAND Flash 都是以晶体管的连接形态命名的，它们是非或、非与电路的简称。Intel 公司于 1988 年开发出 NOR Flash 技术，Toshiba 公司在次年发表了 NAND Flash 结构，Flash 存储器被广泛地运用于简单、快捷的信息存储领域，以及我们所涉及的文件系统中数据存储应用。下面，介绍两类主流 Flash 器件的特性，这是 Flash 文件系统设计和存储空间管理的研究基础。

（1）读写速度

NOR Flash 的随机读取的速率很高, 在 1~4MB 的小容量时具有很高的成本效益, 但是很低的写入和擦除速度大大影响了它的性能。鉴于以上原因, NOR Flash 也称为 Code Flash, 一般作为系统的代码存储介质来使用, 如 BIOS、移动电话和硬盘驱动器的控制内存等都是 NOR Flash 的典型应用。

NAND Flash 最大的特点在于 NAND 结构能提供极高的单元密度, 可以进行高密度存储, 并且写入和擦除的速度非常快。由于其存储密度高, NAND Flash 与 NOR Flash 相比, 单位容量的 NAND Flash 比 NOR Flash 要便宜很多, 因此, NAND Flash 也被称为 Data Flash, 经常作为数码设备的数据存储介质, 如各种移动存储卡、数据存储卡和固态盘式储存介质等。

(2) 性能比较

任何 Flash 器件的写入操作只能在空白单元或已擦除的单元内进行, 所以在进行写入操作之前需要先执行擦除操作。

NAND Flash 执行擦除操作十分简单, 以整块 (block) 为单位进行 (将 “0xff” 写入到要擦除的存储块中); 而 NOR Flash 则是以位 (bit) 为单位进行擦除 (分别将目标块内每一位都写为 “0”)。

NAND Flash 以页 (page) 为单位进行写入操作, 每次写入至少一个页面; NOR Flash 可以按字节写入。

执行一次写入/擦除操作, NAND Flash 最多只需要 4ms (块大小为 8—32K), 而 NOR Flash 需要时间为 5s (块大小为 64—128K)。

(3) 接口比较

NOR Flash 带有 SRAM 接口, 有独立的数据总线和地址总线。NOR Flash 拥有多的地址引脚用来寻址, 可以很容易地存取内部存储空间的每一个字节, 其机制与内存比较相似。因此 NOR Flash 可以片内执行 XIP (eXecute In Place), 应用程序可以直接在 NOR Flash 内运行, 不必把代码读到系统 RAM 中。

NAND Flash 没有采用内存随机读的方式读取存储空间, 其没有专门的地址线据线, 只有控制线和 8 位 I/O 端口来串行存取数据, 而且各个产品或厂商的 Flash 存储器和现有 Flash 文件系统法可能各不相同。绝大多数 NAND 器件只有 8 个 I/O 引脚, 分时传送命令和数据信息, 这一点与硬盘的读写操作有些相似。

(4) 可靠性比较

采用 Flash 作存储介质时需要重点考虑可靠性问题。我们可以从使用寿命、位交换和坏块处理三个方面对 NAND Flash 与 NOR Flash 进行比较。

在 NAND Flash 中每个块的最大擦写次数是一百万次, 而 NOR Flash 的擦写次数是十万次。NAND 存储器除了具有 10 比 1 的块擦除周期优势外, 典型的 NAND 器件的尺寸也要比 NOR 器件小 8 倍。

所有 Flash 器件都受位交换现象的困扰。位交换是指在某些情况下, 存储的数据的某一个比特位会发生反转或被报告反转了。如果只是报告有问题, 重复读操作可以解决反转的问题; 如果某个位真的改变了, 就必须采用错误探测/错误更正 (EDC/ECC) 来进行修复。位交换发生的频率很小, 相比而言, NAND 器件比 NOR 器件发生的频率要高。位反转

对于用 Flash 器件存储多媒体信息时并不是致命的，某一位的错误不足以影响多媒体文件的效果，但如果用来存储操作系统、配置文件或其他敏感信息时，一个小的故障可能导致系统崩溃。

由于制造工艺的不同，NAND Flash 存储器一般在出厂时会有随机分布的坏块，而 NOR Flash 没有这种情况。以前在 NAND Flash 的生产过程中，也曾有过消除坏块的尝试，但是成品率太低，代价太高，额外增加了 Flash 本身的成本。现在对于坏块的处理，通常采用的方法是在使用 NAND 器件前，首先对器件进行初始化扫描以发现坏块，并将坏块作上不能使用的标记。如果不进行这项处理，文件可能被存储在坏块的区域，将导致很高的故障发生率。

通过以上对 NAND Flash 与 NOR Flash 进行的详细比较，我们不难发现，NOR Flash 存储器容量较小，一般用于存放直接执行的代码，其存储数据显得代价太高；NAND Flash 更适合作为更大容量的存储器件，存储多媒体信息数据。所以，本文选择 NAND Flash 存储器作为 Flash 文件系统和空间管理的主要研究对象。

1.3 常见的嵌入式文件系统

鉴于 Flash 存储介质的读写特点，传统适用于 PC 上的文件系统已经不适合应用在嵌入式系统中，基于这样的原因，产生了很多专为 Flash 设备而设计的文件系统，常见的专用于闪存设备的文件系统如下：

（1）线性 Flash 文件系统

线性文件系统又称为连续存放文件系统，每个文件相关的所有信息都连续存放在存储器中，与集中管理的文件系统相比，其优势有：实现更简单，读写更快速，文件系统的关键信息分布存放，安全性较好，任意部分的损坏只影响单个文件，各文件块属性都是平等的。但缺点也很明显，对其他文件系统不兼容，操作效率很低，每次操作都需要从头扫描整个 Flash 块，不容易实现添加，插入、剪切等操作，并且不支持平衡损耗功能。

线性文件系统的设计思路是：文件分为文件头和文件数据区两个部分，每个文件按照顺序存放在 Flash 中，以单向链表来链接文件。文件的起始部分是文件头，包含文件的属性，指向下一个文件头的指针以及文件头和文件数据区的 32 位循环冗余校验码等。

删除文件时，仅仅是简单地把文件头的标识位置 0，同时在碎片整理记录区中记录，表示删除。这样经过多次删除后，就有必要进行碎片整理来回收 Flash 空间，因此碎片整理时还需要在文件系统 Flash 空间的尾部留下一个扇区来做数据备份，必便当碎片整理被打断（如掉电或复位）可以恢复文件系统。这个空闲的扇区必须放在文件系统空间的最后，才能保证文件系统的所有文件占用的 Flash 空间是连续的。

（2）JFFS 文件系统

JFFS 文件系统是瑞典的 Axis Communications 公司在 GNU General Public License 下发布的自由软件，主要用于嵌入式 Linux，相对于 NAND Flash，支持 NOR Flash 的技术更成熟一些。

JFFS 也是日志结构的文件系统，在 Flash 的存储空间中，文件节点链接信息和数据都依次存放于其中。在 JFFS 中，只有一种文件节点 `jffs_raw_inode`，每个这样的节

点都关联到某个文件上，其中包含了一个简单的头部信息、辅助信息以及存储的数据。其中，大的文件可以由很多节点来存放，文件节点的链接信息等特殊的系统信息只占有很少量的空间，通常不会超过一个节点。删除文件的时候，采用的方法是在辅助信息存放的地方设置一个删除在更新文件的时候，新的文件所存放的节点会添加到所有已经使用的存储末尾，旧的文件数据会被废弃，等待在回收空间时重新分配利用。

挂载 JFFS 文件系统时，会把整个存储设备扫描一次，每个存储块都读取一次，然后根据所有节点中存储的信息在控制终端的内存中生成一个文件系统的目录树，同时也自动生成一个 Flash 存储器中所有文件的物理存储位置对应表，用来进行文件的寻址操作。文件目录树的结构通过挂载时生成的信息获得，而读取文件的内容的时候，利用挂载时生成的物理位置表可以轻松地将指定地址的内容读入到缓冲区中以备使用。改变文件的属性和权限等操作，只需要将一个记录新信息的节点更新为改变后的属性即可，旧的节点标上废除标识。改写文件等操作也与此类似，其并不是通过直接改写旧节点来完成的。

在 JFFS 的整个系统中，也包含了与 TrueFFS 中的“碎片回收”相类似的空间回收机制，在操作中，并不会立即对废弃的存储块进行重新利用。当不断积累的废弃存储块越来越多，必然会影响 Flash 存储空间的合理使用，当几乎所有存储块都被占用之后，文件系统就将开始对废弃的存储区块进行回收利用。在进行空间回收时，系统自动从存储块中的第一块开始进行分析，不断将废弃的节点回收，将还在使用的节点进行合并，这样最终合成出一整块废弃的 Flash 存储区块，这样就可以将这一块存储块的内容一次性擦除，成为新的空闲块。

（3）YAFFS 文件系统

YAFFS/YAFFS2 是一种和 JFFSx 类似的闪存文件系统，它是专为嵌入式系统使用 NAND 型闪存而设计的一种日志型文件系统，但 YAFFS 仅仅借鉴了日志系统的思想，不提供日志机能。

和 JFFS2 相比 YAFFS 减少了一些功能，在稳定性上比不上 JFFS2，但是 YAFFS 占用资源较少，所以速度更快，而且对内存的占用比较小。YAFFS 中不需要读取全部文件节点的信息，只在读取某个文件的时候，找文件对应的 ID 号即可，减少了文件系统在加载的时的扫描时间。在 YAFFS 中是从头到尾对块进行搜索，所以在碎片回收方面，JFFS2 的速度较慢，但在损耗均衡性能方面比 YAFFS 要更加出色。

YAFFS 是 NAND Flash 专用的文件系统，充分考虑了其特性，所以在对 NAND Flash 的支持上性能更加出色。YAFFS 自带 NAND 芯片的驱动，并且为嵌入式系统提供了直接访问文件系统的 API，用户可以不使用 Linux 中的 MTD 与 VFS，直接对文件系统操作。YAFFS2 支持大页面的 NAND 设备，并且对大页面的 NAND 设备做了优化。JFFS2 在 NAND 闪存上表现并不稳定，更适合于 NOR 闪存，所以相对大容量的 NAND 闪存，YAFFS 是更好的选择。

1.4 FAT 文件系统简介

FAT 文件系统技术成熟、结构简单、系统资源开销小，易于在不同的硬件平台上实现，文件系统由以下 4 部分组成，按如下顺序排列，如图所示：

系统记录区 SR(System Record)	文件分配表 FAT(File Allocation Table)	文件登记表 FRT(File Register Table)	数据区 (Data Region)
----------------------------	-------------------------------------	-----------------------------------	----------------------

图 1.1 FAT 文件系统结构

0—系统记录区 (System Record Region)

1—FAT 表区 (FAT Region)

2—文件登记表区 (File Register Table Region)

3—数据区 (Data Region)

(1) 系统记录区 (SR, System Record Region)

存放最重要的文件系统信息。Flash 存储器的类型、容量、版本信息、数据区域的位置和大小，还包括分区表引导程序。主引导程序的任务是检查分区表是否正确，自动读取 FAT 表和 FRT 并将其复制到控制终端的内存中。系统记录区存储在存储器的 0 簇 0 扇区，大小为 512Bytes。

(2) 文件分配表 (FAT, File Allocation Table)

存放 Flash 存储器上所有区块的空闲情况以及每个文件的存储连接结构。FAT 文件系统中 有 12 位、16 位、32 位三种不同的格式，不同格式的文件系统管理存储单元的个数不同，如 FAT16 文件系统采用 16 位字长的分配表，最多可以管理 2 的 16 次方 (65536) 的存储单元。文件分配表在存储器中的位置紧跟在系统记录区的后面，大小由存储器中的簇的数目决定，由于文件分配表对文件系统的重要性，所以文件分配表有一个备份，即在原文件分配表地后面在建一个同样的文件分配表。

(3) 文件登记表 (FRT, File Register Table)

文件登记表又叫根目录区 (Root Directory Region)，紧跟在 FAT 表之后，根目录区存放目录项，即每一个文件（目录）的代号、长度、属性、目录、生成或最后修改的时间以及该文件（目录）存储链在文件分配表中的入口。位置从备份的文件分配表尾直到第 0 簇的末尾。

(4) 数据区域 (Data Region) 用于存放数据。从第 1 簇到存储器末尾。

JFFS 和 YAFFS 文件系统是针对 Flash 的特性而设计的文件系统，复杂度比较高，难以在短时间内理解程序的思想，基于这种考虑，本 Flash 文件系统采用 FAT 格式的文件系统。

2 硬件体系结构和软件平台

2.1 硬件体系结构

硬件平台采用的是 UP-NETARM3000 嵌入式实验平台，该平台除了核心模板以外，还包含了系统的外围设备。系统外围设备的硬件部分包括液晶显示屏（LCD 和触摸屏）、USB 通信模块、网络接口模块、键盘、海量 Flash 存储器、系统时钟和日历等。外围设备的硬件部分是保证系统实现指定任务的最底层的部件。丰富外围设备减少了在硬件方面的工作量，方便了文件系统的开发。下面简单介绍一下 UP-NETARM3000 嵌入式实验平台的配置。

- 1) CPU: ARM7TDMI 结构芯片 S3C44B0X, 工作频率 60MHz;
- 2) BIOS: 2M, NOR FLASH(SST39LF160);
- 3) 内存: 8M, SDRAM;
- 4) 海量存储器: 16M, NAND Flash(K9F2808UOC);
- 5) 键盘: 17 键数字小键盘;
- 6) 显示: 320×240 像素、256 色 LCD, 带四线电阻式触摸屏;
- 7) USB 接口: D12 芯片, USB 1.1 标准;
- 8) 串口: 2 路, 最高波特率 115200 b/s;
- 9) 网络接口: RTL8019, 10M 以太网;
- 10) CAN 接口: 波特率 125Kb、250Kb、500Kb、1Mb;
- 11) 调试接口: JTAG, 14 针、20 针;
- 12) 电机: 直流电机（有刷）, 步进电机（带 1/64 减速器）;
- 13) A/D: ARM 自带 8 路 10 位 A /D, 满量程 2.5V, 4 路电位器, 4 路扩展;
- 14) D/A: 10 位, 满量程 4.096V;
- 15) LED: 8 个, 共阴极, 与键盘共同由 ZLG7289 芯片驱动;
- 16) 音频输出: IIS 总线, UDA1341 芯片, 44.1KHz 音频;
- 17) 扩展插槽: 2 个, 提供系统总线以及其他空余资源接口;
- 18) FPGA 扩展板（可选件）: ACEX 1K 系列, 扩展 100I/O, 与系统总线相连;
- 19) GPRS 扩展板（可选件）: SIMCOM 的 SIM100-E 模块, 支持双道语音通信;
- 20) 配套 JTAG 仿真器、电源适配器、通信线缆若干。

在本嵌入式 FAT 文件系统主要用到该平台上的器件有

- 1) CPU: ARM7TDMI 结构芯片 S3C44B0X、
- 2) BIOS: 2M, NOR FLASH(SST39LF160)、
- 3) 内存: 8M, SDRAM、
- 4) 海量存储器: 16M, NAND Flash(K9F2808UOC)、
- 7) USB 接口: D12 芯片, USB1.1 标准
- 8) 串口: 2 路, 最高波特率 115200b/s
- 11) 调试接口: JTAG, 14 针、20 针

S3C44B0X 的寻址空间为 256M，分为 8 个 Bank。在该平台中，Bank0 为 2M 的 NOR FLASH SST39LF160，存储系统初始化程序，物理地址为 0x00000000-0x00100000，Bank1 为 16M 的 NAND Flash K9F2808UOC，做为系统的海量存储器，物理地址为 0x02000000-0x03000000，文件系统存储在该区间。Bank6 为 8M 的 SDRAM，物理地址为 0x0c000000-0x0c800000。系统上电后，PC 指针自动指向 Bank0 的第一个单元，开始进行系统自举，首先完成 S3C44B0X 的初始化程序，将相关程序从 Bank0 复制到 SDRAM 内存中执行。

2.2 NAND Flash 存储器 K9F2808UOC

本实验平台的 NAND Flash 上使用的是 SAMSUNG 的 K9F2808UOC，下面将对此型号的 Nand Flash 的物理结构、管脚描述、与 S3C44B0X 的电路连接作简单介绍，给出向文件系统提供的接口函数读（ReadPage）、写（WritePage）和块擦除的代码。

2.2.1 物理结构

首先我们先从物理结构上来了解这颗芯片，结构图如下所示

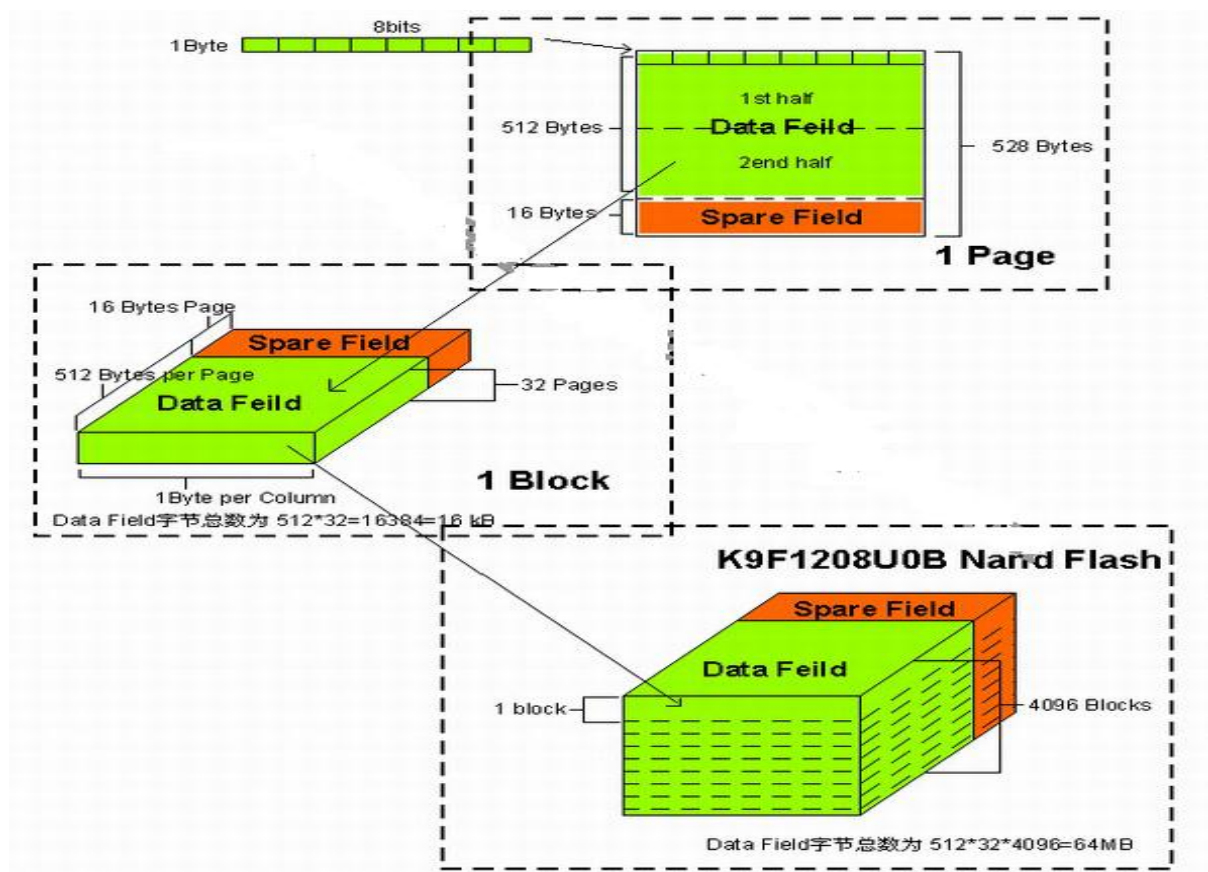


图 2.1 K9F2808UOC 物理结构

正如硬盘的盘片被分为磁道，每个磁道又被分为若干扇区，一块 Nand Flash 被分为若干 Block，每个 Block 又被分为若干 Page。由上图我们可以知道 flash 中 Byte（字

节), Page(页), Block(块) 3 个单位之间的关系为:

1 Page = 512 Bytes Data Field + 16 Bytes Spare Field

1 Block = 32 Pages

K9F2808U0C 总共有 1024 个 Blocks, 故这块 flash 的容量为 $1024 * (32 * 528) = 17302504 \text{ Bytes} = 17\text{MB}$ 但事实上每个 Page 上的最后 16Bytes 是用于存储检验码用的, 并不能存放实际的数据, 所以实际上我们可以操作的芯片容量为 $1024 * (32 * 512) = 16777216 \text{ Bytes} = 16 \text{ MB}$ 。如上图所示, 1 个 Page 总共由 528 Bytes 组成, 这 528 个字节按顺序由上而下以列为单位进行排列 (1 列代表一个 Byte。第 0 行为第 0 Byte, 第 1 行为第 1 Byte, 以此类推, 每个行又由 8 个位组成, 每个位表示 1 个 Byte 里面的 1bit)。这 528Bytes 按功能分为两大部分, 分别是 Data Field 和 Spare Field, 其中 Spare Field 占 528Bytes 里的 16Bytes, 这 16Bytes 是用于在读写操作的时候存放校验码用的, 一般不用做普通数据的存储区, 除去这 16Bytes, 剩下的 512Bytes 便是我们用于存放数据用的 Data Field, 所以一个 Page 上虽然有 528 个 Bytes, 但我们只按 512Bytes 进行容量的计算。Data Field 按位置关系又可分为两个部分, 分别称为 1st half 与 2nd half, 每个 half 各占 256 个 bytes。对 K9F2808U0C 的操作是通过向 Nand Flash 命令寄存器发送命令队列实现的, 下面是 K9F2808U0C 的操作命令集:

表 2.1 K9F2808U0C 的操作命令集

功能	第一个周期	第二个周期	忙时可接受的命令
读 1	00h/ 01h ⁽¹⁾	-	
读 2	50h	-	
读 ID	90h	-	
复位	FFh	-	0
页编程	80h	10h	
块擦除	60h	D0h	
读状态	70h	-	0

向 K9F2808U0C 的命令寄存器发送上图中的某个操作指令后, K9F2808U0C 便知道下一步将要进行的操作, Page 是页面读出的最小单位, 所以地址最小的单位为页号, Block 为 1024 个占 10 位, 每一个 Block 内有 32 个 Page, 占 5 位, 故总的地址传递位数为 15 为, 总线宽度为 8Bit, 需要两个周期。

2.2.2 管脚描述

表 2.2 为 NAND Flash K9F2808U0C 各个管脚的功能。

ALE、CLE 分别为地址锁存使能和命令锁存使能, 当它们为高电平时, 表示命令在 WE 信号的上升沿通过 I/O 口锁存到地址寄存器或命令寄存器中, 所有的数据通过 I/O0-I/O7 输入或输出, 传送到物理地址往往大于 8Bit, 故需要分为多次传送。一般在对 Flash 进行读写操作之前要检查 Flash 是否处于忙状态, Flash 通过管脚 R/B 表示

当前工作状态是否忙，为高表示处于空闲状态。

表 2.2 K9F2808U0C 管脚介绍

管脚名称	管脚功能
I/O0-I/O7	数据输入/输出 这些 I/O 口用来输入命令地址和数据以及在读操作时输出数据，当芯片未被选择使用或输出禁止时 I/O 口为高阻态。
CLE	命令锁存使能 CLE 输入控制着发送到命令寄存器的命令的有效通路，当它为有效的高电平时命令在 WE 信号的上升沿通过 I/O 口锁存到命令寄存器中。
ALE	地址锁存使能 ALE 输入控制着地址到内部地址寄存器的有效路径，ALE 为高电平时地址在 WE 信号的上升沿被锁存。
CE	芯片使能 CE 输入是器件选择控制信号，当器件出于忙状态时 CE 的高电平被忽略，器件在执行编程或擦除操作时不会返回到等待模式。
RE	读使能 RE 输入控制着串行数据的输出，当该信号有效时数据被驱动到 I/O 总线上，RE 变成上升沿 t_{REA} 后数据有效，RE 每出现一次上升沿内部列地址计数器就加 1。
WE	写使能 WE 输入控制着写 I/O 口操作命令地址和数据，在 WE 脉冲的上升沿被锁存。
WP	写保护 WP 为电源变化时的无意写提供了写/擦除保护，当 WP 管脚为有效低电平时内部高电压发生器复位。
R/B	读/忙输出 R/B 输出用来指示器件的工作状态，为低时表明器件正在执行编程擦除或随机读操作，这些操作完成后 R/B 返回高电平该信号，是一个开漏输出，当芯片未被选择使用或输出被禁能时它不呈现高阻态。

2.2.3 电路连接

图 2.2 是 S3C44BOX 与 NAND Flash K9F2808U0C 的电路连接图：

通过下图可以看到，I/O0-I/O7 接 S3C44BOX 的地址线 ADDR0-ADDR7，ALE、CLE 接 S3C44BOX 的 C 端口的 10 号和 11 号，R/B 接 E 端口 0 号，S3C44BOX 的 nGCS2 与 nWE 信号相与接 K9F2808U0C 的 we 管脚，nGCS2 与 nRE 信号相与接 K9F2808U0C 的 re 管脚。了解电路的连接，对于 K9F2808U0C 的读写程序的理解很重要。

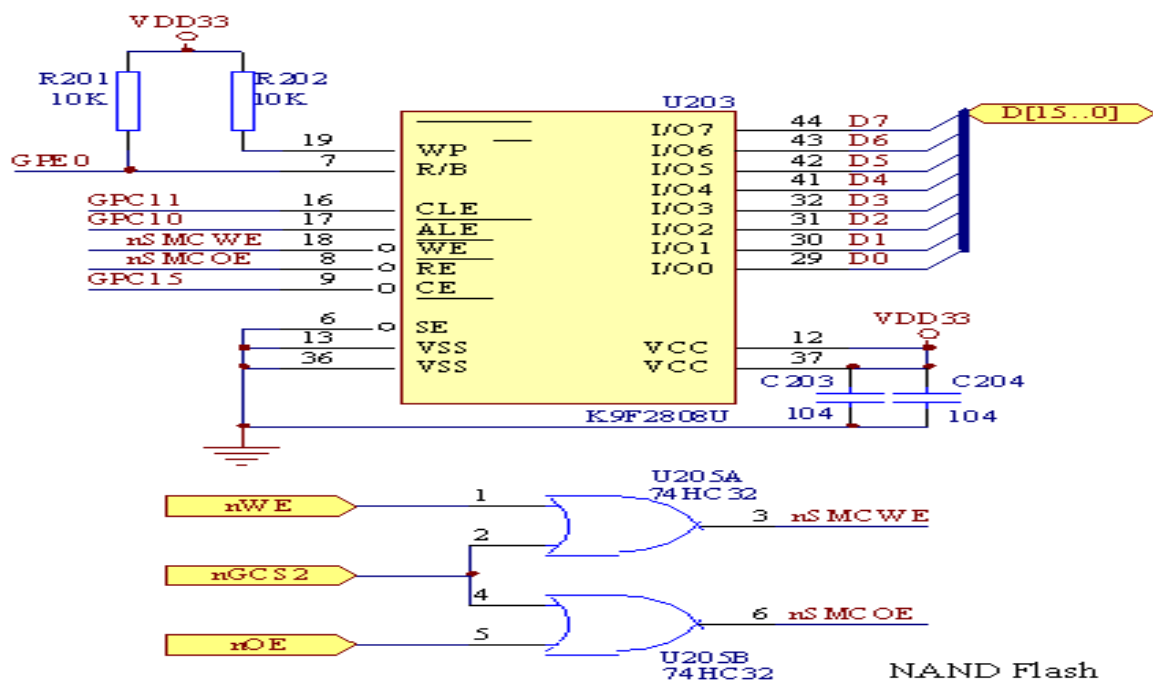


图 2.2 S3C44BOX 与 K9F2808U0C 电路连接图

注释：

- ◆ GPE0：E 端口 PE0；
- ◆ GPC10：C 端口 PC10；
- ◆ GPC11：C 端口 PC11；
- ◆ GPC15：C 端口 PC15；
- ◆ D0-D7：地址线 ADDR0-ADDR7；
- ◆ nGCS2：片选型号，为高是表示地址、数据和命令传送到 Bank1；
- ◆ nRE：读允许信号；
- ◆ nWE：写允许信号。

2.2.4 读写程序

NAND Flash 的读写操作以 Page 为单位，向文件系统提供的主要接口有页的读程序 ReadPage、页的写程序 WritePage、块擦除程序 Erase_Cluster。下面给出程序的代码及相应注释，对程序的理解要结合表 2.1 Flash K9F2808U0C 的操作命令集、表 2.2 K9F2808U0C 管脚介绍、图 2.2 S3C44BOX 与 K9F2808U0C 电路连接图。

程序清单2.1 与程序相关的宏定义

```
#define RB (0x001) (1)
#define CLE (0x800)
#define ALE (0x400)
#define CE (0x8000)
#define Nand_IO (*(volatile unsigned char *)0x04000000) (2)
```

```
#define FC_CMD {rPDATC |= CLE;rPDATC &= ~(ALE|CE);}
#define FC_ADDR {rPDATC |= ALE;rPDATC&=~(CLE|CE);}
#define FC_DATA {rPDATC &= ~(ALE|CLE|CE);}
#define FC_INACTIVE {rPDATC |= CE;rPDATC&=~(ALE|CLE);}
#define WAITRB while(!(rPDATC & RB))
#define KM29U128_ID (0xec73)
#define WRITEVERIFY (0)
#define NandPageSize 528
```

rPDATC 为 PC 口数据寄存器，当端口定义为输入口时，这个寄存器对应着引脚的状态，当端口定义为输出口时，输出引脚的状态与该引脚相同。此处 PC 口均定义为输出口，用二进制表示时，宏 RB、CLE、ALE、CE 分别表示第 0 位、第 11 位、第 10 位、第 15 位为 1，其余位为 0 的二进制数 [2.1(1)]，其值与 rPDATC 或 rPDATC 相或表示将端口 PC 或 PE 的相应位置为高，相与表示相应位保持不变其余均置为低电平，用于向 NAND Flash 发送控制信号，表示要传送地址、指令、数据或芯片使能等操作。宏 Nand_IO 地址指向 Bank1 [2.1(2)]，当程序中形式为 Nand_IO=变量，编译器会识别为写指令，nGCS2 和 nWE 为高电平，数据通过 ADDR0-ADDR7 写入 NAND Flash 数据寄存器，同理，形式为 变量=Nand_IO，识别为读指令，nGCS2 和 nRE 为高电平，数据通过 ADDR0-ADDR7 读入 SDRAM 的缓冲器中。WAITRB 不断查询 NAND Flash 的忙标志，一直等到 NAND Flash 处于空闲状态 [2.1(3)]。

程序清单 2.2 页的读程序

```
void ReadPage(unsigned int block,unsigned int page,unsigned char *pPage)
{
    int i;
    unsigned int blockPage = (block<<5) + page;
    FC_CMD;
    Nand_IO = 0x00;
    FC_ADDR;
    Nand_IO = 0;
    Nand_IO = blockPage & 0xff;
    Nand_IO = (blockPage>>8) & 0xff;
    for(i=0;i<3;i++);
    FC_DATA;
    WAITRB;
    for(i=0;i<528;i++)
        { *pPage++=Nand_IO; }
    FC_INACTIVE;
}
```

NAND Flash K9F2808UOC 最小的读取单位为 Page。blockPage 为地址共 15 位，高 10 为块号（1024 个 Block），低 5 位为页号（每页 32 个 Page）。宏 FC_CMD 使 CLE 为高电平，K9F2808UOC 的命令锁存使能 [2.2(1)]，Nand_IO = 0x00 将 0x00 写入命令寄存器，表示要进行读操作 [2.2(3)]。宏 FC_ADDR 使 ALE 为高电平，K9F2808UOC 的地址锁存器使能，Nand_IO = 0 表示从 1st half 开始读 [2.2(4)]，blockPage 为 15

位，总线宽度为 8 位，需要分两次传送[2.2(5)]。FC_DATA 使信号 ALE、CLE、EC 为低电平，在每个 WE 的上升沿将一个 8Bit 的数据送到 K9F2808U0C 的数据寄存器[2.2(6)]。

程序清单2.3 页的写程序

```
int WritePage(unsigned int block,unsigned int page,unsigned char *pPage)
{
    unsigned int blockPage = (block<<5) + page;
    int i;
    U8 *Source;
    Source=pPage;
    FC_CMD;                                     (1)
    Nand_IO = 0x80;                             (2)
    FC_ADDR;                                    (3)
    Nand_IO = 0;                                (4)
    Nand_IO = blockPage & 0xff;
    Nand_IO = (blockPage>>8) & 0xff;
    FC_DATA;                                     (5)
    for(i=0;i<528;i++)
        {Nand_IO=*pPage++;}
    FC_CMD;
    Nand_IO = 0x10;                             (6)
    for(i=0;i<10;i++);
    WAITRB;//wait max 500us;
    Nand_IO = 0x70;
    FC_DATA;
    for(i=0;i<3;i++); //twhr=60ns
    if((Nand_IO & 0x1))
    {
        FC_INACTIVE;
        return 0;
    }
    else
    {
        FC_INACTIVE;
        pPage = Source;
        #if (WRITEVERIFY==1)
            return VerifyPage(block,page,pPage);
        #else
            return 1;
        #endif
    }
}
```

FC_CMD 使 K9F2808U0C 的指令锁存器使能[2.3(1)]，表示数据要送到 K9F2808U0C 的指令寄存器。0x80 表示指令为写页[2.3(2)]，FC_ADDR 使 K9F2808U0C 的地址寄存器使能[2.3(3)]，发送页地址[2.3(4)]，0 表示从页的 1st half 开始，blockPage 长为 15Bit，分两次传送，Nand_IO = 0x10 为块擦除的第二个周期[2.3(6)]。

程序清单2.4 块擦除程序

```

unsigned char Erase_Cluster(unsigned int cluster)
{
    unsigned int blockPage = (cluster<<5);           (1)
    int i;
    FC_CMD;                                           (2)
    Nand_IO = 0x60;                                   (3)
    FC_ADDR;                                           (4)
    Nand_IO = blockPage & 0xff;                       (5)
    Nand_IO = (blockPage>>8) & 0xff;
    for(i=0;i<2;i++); //tWC 50ns
    FC_CMD;
    Nand_IO = 0xd0;                                   (6)
    for(i=0;i<3;i++);
    FC_DATA;
    WAITRB; //wait max 3ms                            (7)
    FC_CMD;
    Nand_IO = 0x70;                                   (8)
    FC_DATA;
    if (Nand_IO & 0x1) //erase error
    {
        FC_INACTIVE;
        return FAIL;
    }
    else
    {
        FC_INACTIVE;
        return OK;
    }
}

```

NAND Flash 擦除的基本单位是块，K9F2808UOC 共有 1024 个块（每个块 16K）需要 10 位二进制数，左移 5 位后即为该块的起始页号 [2.3(1)]。FC_CMD 使 CLE 为高电平通知 K9F2808UOC 将向命令寄存器发送命令 [2.3(2)]，擦除操作第一周期发送的数为 0x60 [2.3(3)]，第二周期为 0xd0 [2.3(6)]。blockPage 为 15 位分两次传送 [2.3(5)]。WAITRB 不断查询 R\B（忙信号）直到 K9F2808UOC 处于空闲状态 [2.3(7)]。发送指令 0x70 表明程序要读取状态信息判断擦除是否成功 [2.3(8)]。

2.3 软件平台

在软件平台的选择上，现在有很多的嵌入式开发软件平台，比如 ARM 公司的 ADS 系列的软件，还有 IAR 公司的 IAR Embedded Workbench for ARM Evaluation 等软件。

ARM ADS 全称为 ARM Developer Suite，是 ARM 公司为方便用户在 ARM 芯片上进行应用软件开发而推出的一套集成开发工具。ARM 公司是 ARM 系列芯片的设计商，ADS

的设计充分考虑了 ARM 系列芯片本身的特点，编译生成的代码在 ARM 系列芯片上运行具有较高的效率。ARM ADS 除了可以安装在 Windows NT4, Windows 2000 操作系统下，还支持 Windows XP 操作系统。ADS 由命令行开发工具, ARM 实时库, GUI 开发环境(Code Warrior 和 AXD) ,实用程序和支持软件组成。有了这些部件，用户就可以为 ARM 系统的 RISC 处理器编写和调试自己的开发应用程序了。

ADS1.2 是一套 ARM 汇编、C 和 C++的集成开发环境，包括了几个有用的开发工具：

A. CodeWarrior IDE for the ARM Developer Suite

自身集成的 CodeWarrior for ARM 是一套完整的开发工具，充分发挥了 ARM RISC 的优势，使产品开发人员能够很好的应用尖端的片上系统技术。该工具是专为基于 ARM RISC 的处理器而设计的，它可加速并简化嵌入式开发过程的每一个环节，使得开发人员只需要一个集成软件开发环境就能研制出 ARM 产品，在整个开发周期中，开发人员无需离开 CodeWarrior 开发环境，因此节省了在操作工具上花的时间，使的开发人员有更多的精力投入到代码编写上来。

B. AXD Debugger

程序员可以通过这个软件使用 debug agent 可以对包含有调试信息的，正在运行的可以执行代码进行比如变量的查看，断点的控制等调试操作，这样的话，使的找程序的错误变得更加的简单，快速。

在程序的设计中还用到了其他的软件，一个优秀的软件可以大大加快程序的开发，如用 UarmJtag2.0 或实现 Flash Programmer 向 Nor Flash 烧写程序功能，串口通信软件 Sscom 或可以方便实现 PC 与嵌入式平台的串口通信，在一些特殊情况下也可以完成串行通信功能，程序编辑软件 Source Insight 在编辑大量的代码时使程序的层次更加清晰，在阅读代码时减少了来回查找函数的麻烦。

3 FAT 文件系统总体设计

3.1 硬件设计

在将嵌入式操作系统移植到 ARM 嵌入式开发平台时参考 FAT16 为该系统扩展了一个简单的文件系统，从而使该操作系统功能更强大，也符合实际嵌入式产品开发的需要。开发平台的硬件中有一片容为 16M 的 NAND FLASH 存储芯片 K9f2808U0C 作为嵌入式设备的固态数据存储器，或称为电子硬盘。该存储器由文件系统管理，在文件系统的功能函数与 FLASH 芯片之间有相关驱动程序实现高层系统功能和底层具体硬件的数据交换。

嵌入式系统经常使用的存储介质 FLASH 芯片具有和磁盘完全不同的结构特点，开发平台上的 K9F2808U0C 芯片具有 1024 个 Block，每个 Block 有 32 个 Page，每个 Page 有 $512+16=528$ 个 Byte。这种芯片的读写操作具有以下特点：

- 1) 必须以 Page 为单位进行读写；
- 2) 写之前必须先擦除原有内容；
- 3) 擦除操作必须对 Block 进行，即一次至少擦除一个 Block 的内容。

针对这种情况，将一个 Page 定为一个扇区，将 1 个 Block 即 32 个扇区定为一个簇，簇的容量正好为 $512 \times 32 = 16K$ ，满足 FAT16 对簇的要求。FAT 是按簇为单位分配空间的，但其给出的地址 LBA(Logical Block Address)只是一个扇区号，比如 $LBA=0 \times 40$ 对于 FLASH 芯片来说实际代表簇 2 的扇区 0。因此需要将 LBA 转换为芯片的物理地址才可以进行数据存取操作，转换关系如下：

FLASH 的 Block = $LBA / 0 \times 20$;

FLASH 的 Page = $LBA \% 0 \times 20$ 。

根据该公式就可以将 LBA 转换为正确的 Block 和 Page，然后就可以用芯片的读写命令存取该 Block/Page 的数据了。

FAT 文件系统包括 MBR 区，FAT 区，DIR 区，DATA 区。其中 MBR（主引导记录）区在磁盘文件系统中也称 0 扇区。MBR 扇区必须包括 BPB（BIOS Parameter Block），其中有对文件系统进行识别的关键信息。由于开发平台的电子硬盘结构比较简单并且不求启动，MBR 不必含有启动代码，但最后两个字节必须是 $0 \times 55AA$ 。FAT 区存放文件分配表的相关信息，对文件内容的增减会引起 FAT 表中数据的变化。DIR 区是根目录区，存放根目录的内容。0 扇区以后知道最后一个簇为 DATA 区即数据区，存放目录和文件内的内容。下表为 MBR 区，FAT 区，DIR 区在 0 扇区的分布。

表 3.1 FLASH 的 Block0 的内容

LBA	Block	Page	Bytes	说明
0	0	0	512	MBR=BPB+Executable Code+ $0 \times 55AA$
1-4	0	1-4	2048	第 1 个 FAT 区
5-8	0	5-8	2048	第 2 个 FAT 区（备份）
9-31	0	9-31		DIR 区，BPB 中的目录项数使其正好添满本簇。
32	2	0		DATA 区起始扇区。

计算过程如下：

MBR 占 512Bytes，位于 0Block、0Page。一个 Block 的大小为 16KB，存储器共有 1024 个 Block，FAT 表中每个元素对应一个 Block 号，为了表示出全部的 Block 号，需要 10 位二进制数，在内存中要占 2Bytes，FAT 表达大小为 $1024 \times 2 = 2048\text{Bytes}$ ，即为 4 个 Page（每个 Page 512KB），FAT 表记录了文件系统的关键数据，如 FAT 表达数据受到破坏整个文件系统将崩溃，故另外一个 FAT 表紧跟在第一个 FAT 表之后用作备份，故 FAT 表共占用了 8 个 Page。0Block 的其余 Pages 为根目录区，共 $32 - 9 = 23\text{Pages}$ 。

在系统对 FLASH 芯片进行格式化操作时将 FAT 信息写入芯片，使其逻辑空间符合文件系统的需要。当主机发出 READ 命令后，FLASH 读写操作开始，首先读取 MBR 得到诸如扇区大小、每簇扇区数、总扇区数等存储介质有关信息。然后读取 FAT 得到文件目录列表，基于此就可以进行文件的复制、删除、创建等操作了。

3.2 用户交互

为了验证程序的运行是否正确，需要用户和文件系统进行交互，嵌入式平台与 PC 之间通信的方式主要有三种：并口、串口、网口。并口多用于程序下载。在本文件系统中嵌入式平台和 PC 之间通过串口进行通信，由 PC 发送文件操作的指令到嵌入式平台，在嵌入式平台上执行，执行后的结果再通过串口发送为用户，指令的格式模仿 DOS 格式，各命令的格式及对应的功能如下：

表 3.2 指令格式

格式(指令 参数 1 参数 2)	对应的功能
cf(crfile) filename	创建一个文件
md(mkdir) dirname	创建一个文件夹
move path1 path2	移动文件
cd(chdir) path	改变路径
copy path1 path2	将一个文件复制到另一个位置
del filename	删除文件
dir	显示一个目录的文件和子目录
rd(rmdir) dirname	删除文件夹
ren(rename) oldn newn	文件重命名
rf(rdfile)	读文件
open filename	打开文件
wf(wtfile)	写文件
format	格式化文件系统
help	显示菜单
exit	退出系统

3.3 指令识别

程序中对文件系统的操作主要通过串口发送指令，指令以单个字符的方式发送，在程序中连接成字符串，需要对字符串进行处理将指令和参数分开，分别存放在不同的字符串数组中。

程序清单3.1 字符串分割

```
char currentpath[15]="\\";
char command[8];
char first[20];
char second[20];
int formatcom(const char* fullcom)
{
    char i=0,j=0;
    if(fullcom==NULL)
        return 0;
    while(fullcom[i]!='\0'&&fullcom[i]!=' ')
    {
        command[j++]=fullcom[i++];
    }
    command[j]='\0';
    while(fullcom[i]==' ')
        i++;
    j=0;
    while(fullcom[i]!='\0'&&fullcom[i]!=' ')
    {
        first[j++]=fullcom[i++];
    }
    first[j]='\0';
    while(fullcom[i]==' ')
        i++;
    j=0;
    while(fullcom[i]!='\0'&&fullcom[i]!=' ')
    {
        second[j++]=fullcom[i++];
    }
    second[j]='\0';
}
```

字符数组 currentpath, command, first, second 为全局变量。fullcom 为通过串口传送的指令，函数将参数 fullcom 按空格分为三个字符串，分别赋值给 command, first, second, 代表指令、参数 1、参数 2。如没有参数则参数 1、参数 2 为空。

字符串 command 存放的是指令部分，程序将和表 3.2 中的各个指令对比直到找到相同的，然后调用相应的文件系统 API 函数。实现的流程图如：

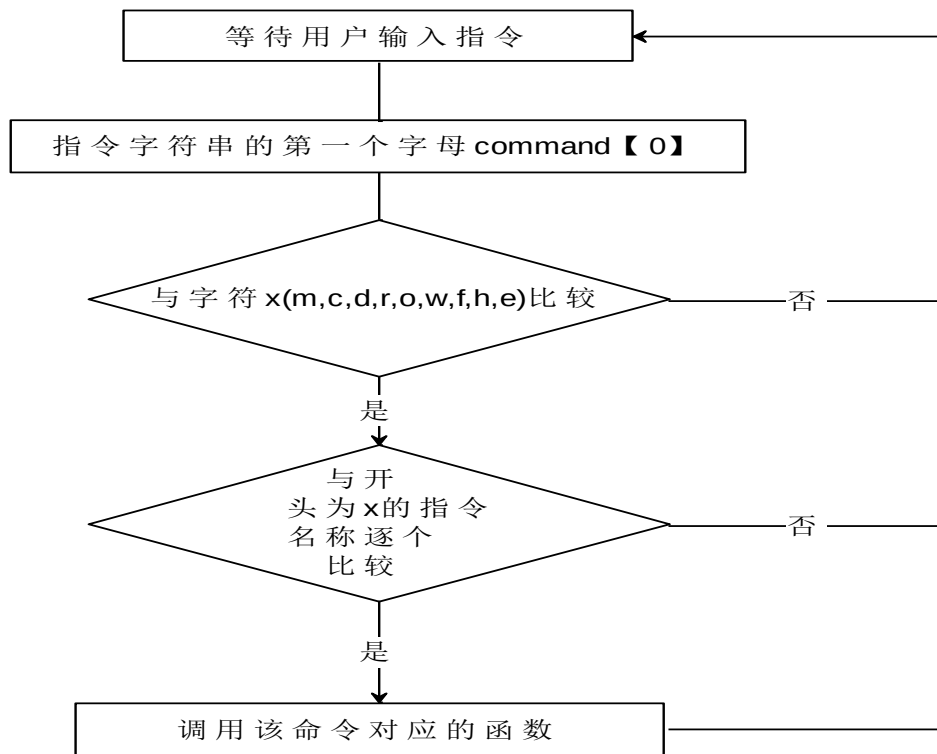


图 3.1 指令流程图

3.4 功能流程图

程序执行的流程图如下：

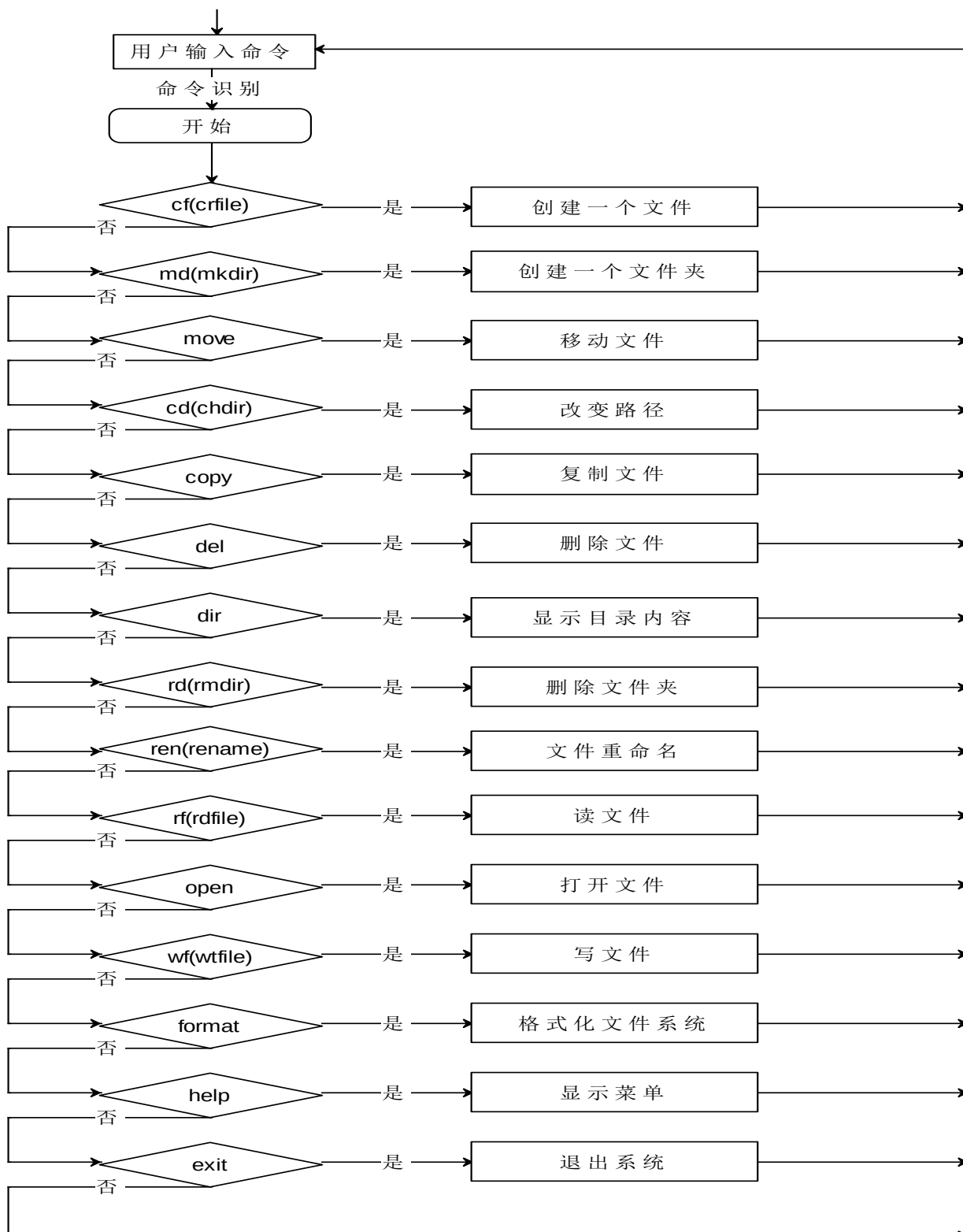


图 3.2 功能流程图

4 FAT 详细设计及代码实现

详细设计的工作包括处理过程和代码设计，即根据总体设计的每个功能模块的要求，利用相应的开发工具设计出应用系统的过程。由于篇幅的限制，在这里不给出全部的程序编码，只给出程序核心数据结构和在 FAT API 函数中调用频率较高的函数。

4.1 重要文件结构

为了便于对 FAT 文件系统的理解，先介绍相关的两个重要的结构。

4.1.1 FAT 表结构

FAT 文件系统的文件管理采取了显式链接的方式，就是把用于链接文件的每个物理块的指针，显式的存放在内存的一张链接表中，表的序号是物理盘号，从 0 开始直至 $N-1$ ， N 为物理盘块总数。在每个表项中存放链接指针，即下一盘块号。在该表中，凡是属于某一文件首指针所对应的盘块号，均作为文件地址被填入相应文件的 FCB 的物理地址字段中。由于查找纪录的过程是在内存中进行的，提高了检索的速度。在该文件系统中每个文件对应一个 DIRENTRY 数据结构，在 DIRENTRY 结构的成员 deStartCluster 表明了文件（目录）的起始簇号，在 FAT 表中以此查找后续的簇号，实现对文件内容的操作。

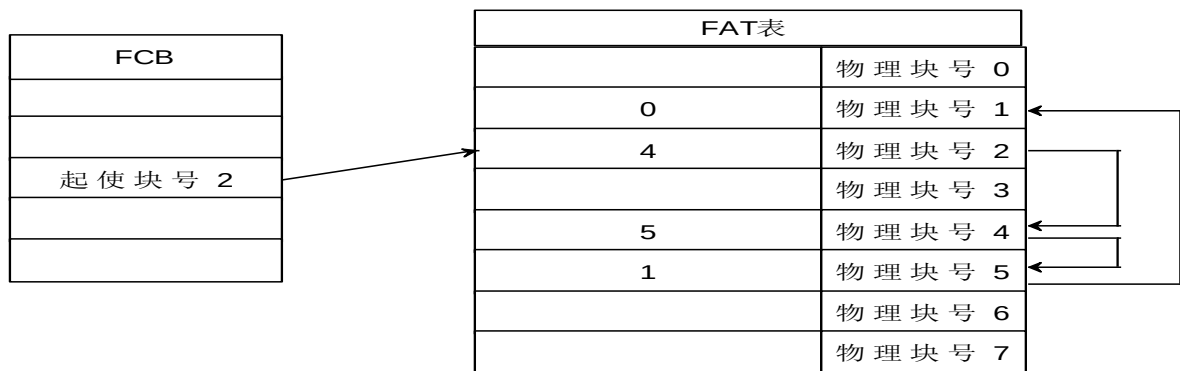


图 4.1 FAT 表结构

图中的文件一共占用了 3 个盘块，依次为 2，4，5。

4.1.2 目录树结构的实现

在 FAT 文件系统中，目录和文件使用同样的数据结构即 DIRENTRY 表示，DIRENTRY 中包含文件相关的信息，子目录的 DIRENTRY 结构存储在父目录中，DIRENTRY 的大小为 16Bytes，1 个扇区中可以包含 32 个 DIRENTRY 数据结构，1 个簇中可以包含有 1024 个 DIRENTRY 数据结构，所有的父目录和子目录形成了树形结构，如下图所示。

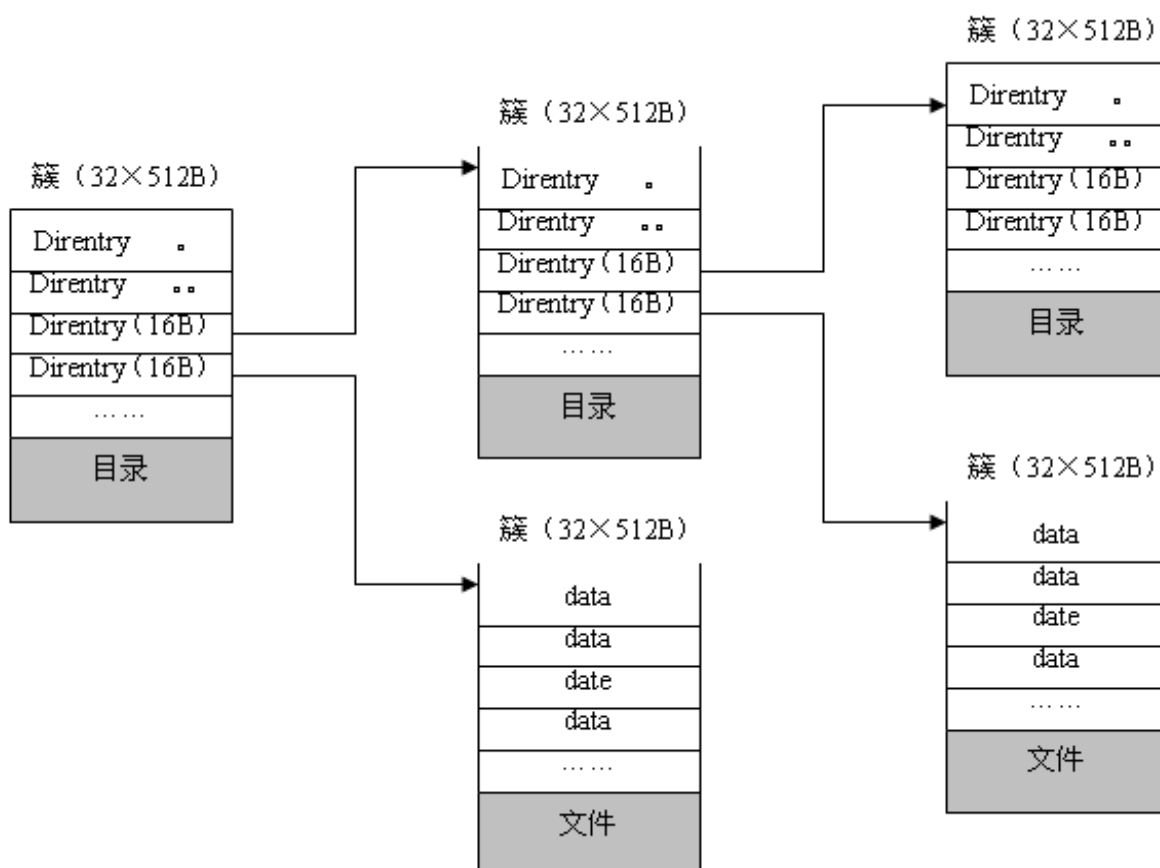


图 4.2 目录结构

4.2 核心数据结构

下面是 FAT 文件系统的数据结构 DIRENTRY、_FILE、BPB50、EXTBOOT、BOOTSECTOR50。

程序清单4.1 宏定义

```
#ifndef BYTE
typedef unsigned char BYTE;
typedef unsigned char U8;
#endif
#ifndef CHAR
typedef char CHAR;
#endif
#ifndef WORD
```

```
typedef unsigned short WORD;
#endif
#ifndef DWORD
typedef unsigned long DWORD;
#endif
```

为了便于程序理解将宏定义列出。

程序清单4.2 重要的全局变量定义

```
unsigned int  FirstDataSector;
unsigned int  BytesPerSector;
unsigned int  FATsectors;
unsigned int  SectorsPerCluster;
unsigned int  FirstFATSector;
unsigned int  FirstDirSector;
DWORD RootDirSectors; // Numbers of sectors occupied by Root Directory.
DWORD RootDirCount;
WORD FatCache[FAT_SIZEOFWORD];
BYTE SectorBuffer[512]; //SectorBuffer
BYTE ClusterBuffer[32][512]; //ClusterBuffer
```

- ◆ FirstDataSector: 第一个数据区所在的扇区号 (32);
- ◆ BytesPerSector: 每个扇区的字节数 (512);
- ◆ FATsectors: FAT 表所占的扇区数 (4);
- ◆ SectorsPerCluster: 每个簇的扇区数 (32);
- ◆ FirstDirSector: 第一个目录项所在的扇区号 (9);
- ◆ RootDirSectors: 根目录区占用的扇区数 (28);
- ◆ DWORD RootDirCount: 根目录中能记录的最多的目录信息 (368);
- ◆ FatCache: FAT 信息在内存中的缓冲区;
- ◆ SectorBuffer: 页面数据在内存中的缓冲区 (512Bytes);
- ◆ ClusterBuffer: 簇在内存中的缓冲区 (32×512 Bytes)。

括号内为变量的值。

程序清单4.3 与FAT文件参数信息相关的数据结构

```
typedef __packed struct {
    WORD    bpbBytesPerSec; //每扇区字节数
    BYTE    bpbSecPerClust; //每簇的扇区数
    WORD    bpbResSectors; //保留区域中的保留扇区数
    BYTE    bpbFATs; //此卷FAT结构的份数，必须为2
    WORD    bpbRootDirEnts; //根目录项数（每项长度为32字节）
    WORD    bpbSectors; //存储器上的扇区总数
    BYTE    bpbMedia; //对于固定存储介质使用0xF8,对于移动存储介质使用F0。
    WORD    bpbFATsecs; // FAT每个FAT分区所占的扇区总数。
    WORD    bpbSecPerTrack; //每道的扇区数，对于非磁头、柱面、扇区结构的介质，此域可以为0。
    WORD    bpbHeads; //磁头数，对于非磁头、柱面、扇区结构的介质，此域可以为0。
    DWORD    bpbHiddenSecs; //FAT表所在的分区前面的隐藏扇区数，，对于非分区介质可
```

以为0。

```

        DWORD    bpbHugeSectors; //扇区总数
    }BPB50;

typedef __packed struct {
    CHAR    exDriveNumber;        //操作系统有关参数，软盘使用0x00,硬盘使用0x80
    CHAR    exReserved1;         // reserved保留
    CHAR    exBootSignature;     //扩展引导标记(0x29)
#define EXBOOTSIG    0x29
    DWORD    exVolumeID;         //卷的序列号。
    CHAR    exVolumeLabel[11];   //1个字节长度卷标，此域需要与目录中的 卷标一致。
    CHAR    exFileSysType[8];    //标志
}EXTBOOT;

typedef __packed struct {
    BYTE    bsJump[3];           // jump inst E9xxxx or EBxx90
    CHAR    bsOemName[8];        // OEM name and version
    CHAR    bsBPB[25];           // BIOS parameter block
    CHAR    bsExt[26];           // Bootsector Extension
    CHAR    bsBootCode[448];     // pad so structure is 512b
    BYTE    bsBootSectSig0;      // boot sector signature byte 0x55
    BYTE    bsBootSectSig1;      // boot sector signature byte 0xAA
#define BOOTSIG0    0x55
#define BOOTSIG1    0xaa
}BOOTSECTOR50;

```

BPB50 大小为 15Bytes、EXTBOOT 大小为 26Bytes、BOOTSECTOR50 的大小为 512Bytes。数据结构 BOOTSECTOR50 主要在文件系统的格式化时使用，各个成员被赋予文件系统的相关参数，然后由内存写入 0 扇区。在格式化过程中 BPB50、EXTBOOT 分别和 BOOTSECTOR50 的成员 bsBPB[25]、bsExt[26]相对应。

程序清单4.4 重要的数据结构

```

typedef __packed struct
{
    BYTE    deName[8];           //文件（目录）名
#define SLOT_EMPTY    0x00        // 文件属性为隐藏
#define SLOT_DELETED    0xe5    // 文件已删除标记
#define IS_DIR        0x2e        // 文件属性为目录
    char    deExtension[3];      //文件扩展名
    BYTE    deAttributes;        //文件属性
#define ATTR_NORMAL    0x00        // 普通文件
#define ATTR_READONLY    0x01    //只读文件
#define ATTR_HIDDEN    0x02        //隐藏文件
#define ATTR_SYSTEM    0x04        //系统文件
#define ATTR_VOLUME    0x08        //卷标
#define ATTR_LONG_FILENAME    0x0f //长文件名

```



```

#define ATTR_DIRECTORY 0x10          //目录文件
#define ATTR_ARCHIVE 0x20           //新建文件或修改后的文件
        BYTE          deLowerCase;   //字符为小写
#define LCASE_BASE 0x08             //文件（目录）名为小写字母
#define LCASE_EXT 0x10              //文件扩展名为小写字母
        BYTE          CrtTimeTenth;  // hundredth of seconds in CTime
        WORD          CrtTime;       //创建时间
        WORD          CrtDate;       //创建日期
        WORD          LstAccDate;     //访问时间
        WORD          WrtTime;       //修改时间
        WORD          WrtDate;       //修改日期
        WORD          deStartCluster; //文件（目录）起始簇号
        DWORD         deFileSize;    //文件大小
}DIRENTRY;

typedef __packed struct
{
    int valid;                //1 valid, 0 free.文件状态空闲或正在被访问
    DWORD DirSectorNum;       //文件的DIRENTRY在父目录的扇区号。
    int DirIndex;             //文件的DIRENTRY在父目录的扇区内地相对位置
    DWORD StartSectorNum;     //文件的起始扇区号
    DWORD CurrentSectorNum;   //当前所在扇区号
    DWORD SectorOffset;       //扇区偏移数
    DIRENTRY dir;             //文件DIRENTRY数据结构
    unsigned long offset;     //文件当前读指针相对起始偏移的字节数
}_FILE;

```

在 FAT 文件系统中，目录和文件使用同样的数据结构即 DIRENTRY 表示，由文件属性 deAttributes 的值来标识目录或文件。DIRENTRY 占 32Bytes，是文件的一个重要标识，一个扇区(512Bytes)中最多可有 16 个 DIRENTRY 数据结构，deName[0]=0xe5 标识文件被删除，deName 的修改表示文件被重命名，文件的查找是在父目录的簇中找到 deName 与文件名相同的 DIRENTRY 数据结构，文件复制、移动实质是文件对应 DIRENTRY 数据结构的复制或删除，DIRENTRY 数据结构中有文件的起始簇号等重要信息，文件的读写首先要在文件的父目录中找到文件的 DIRENTRY 数据结构，然后根据文件 DIRENTRY 数据结构中的起始簇号 deStartCluster 在 FATCache 中查找文件的所有簇，在对文件或目录的一系列操作中 DIRENTRY 数据结构是一个重要的角色。除根目录外每个文件或目录都有对应的 DIRENTRY 数据结构，每个文件或目录的 DIRENTRY 数据结构都存储在其父目录的某个簇中。

_FILE 在根据文件路径查找文件或文件读写是有重要的作用，根据 _FILE 的 DirSectorNum、DirIndex 可以定位文件的 DIRENTRY 数据结构在父目录的位置，从而可以删除、更新、创建文件。文件的读写中 _FILE 中成员 CurrentSectorNum 是单前文件的读写指针所指向的扇区，SectorOffset 是读写指针在扇区的偏移，offset 是文

件读写指针相对于文件起始偏移的字节数。函数中常使用类型为 `_FILE` 的变量作参数，将文件（目录）在父目录中的 `DIRENTRY` 数据结构复制到变量的 `dir` 成员，从而在程序中对文件（目录）的 `DIRENTRY` 数据结构进行操作。

4.3 重要的函数

文件系统 API 函数是通过反复调用一些函数来实现的，其中部分函数调用的频率比较高，现将程序及注释列出，其他相对简单的函数给出了各个函数的功能。

程序清单4.5 读取扇区

```
DWORD CurrentCacheSector;
BYTE* GetSectorData(DWORD StartSector)//LBA
{
    unsigned int block;
    unsigned int page;
    if((CurrentCacheSector == StartSector) && (StartSector != 0))
        return SectorBuffer;
    block=StartSector/0x20;                                (1)
    page=StartSector%0x20;
    ReadPage(block,page,SectorBuffer);                    (2)
    CurrentCacheSector = StartSector;
    return SectorBuffer;
}
```

`CurrentCacheSector` 为全局变量，表示当前在 `SectorBuffer` 中数据所在扇区号，除该函数外，在 `Flush()` 中也用到该变量。`StartSector` 为要读取的扇区号，计算出它所在的簇号和该簇的扇区号 [4.5(1)]，调用 `ReadPage()` 函数将数据读入内存缓冲区 `SectorBuffer` 中 [4.5(2)]。

程序清单4.6 更新簇

```
void Flush()
{
    int i,j;
    unsigned int block;
    unsigned int page;
    block=CurrentCacheSector/0x20;
    page =CurrentCacheSector%0x20;
    memcpy(ClusterBuffer[page],SectorBuffer, BytesPerSector);
    if(page==0)
    {
        for(i=1;i<SectorsPerCluster;i++)
        {
            ReadPage(block,i,SectorBuffer);
            memcpy(ClusterBuffer[i], SectorBuffer, BytesPerSector);
        }
    }
    else
    {
        for(i=0;i<page-1;i++)
```

```

    {
        ReadPage(block,i,SectorBuffer);
        memcpy(ClusterBuffer[i], SectorBuffer, BytesPerSector);
    }
    for(i=page+1;i< SectorsPerCluster;i++)
    {
        ReadPage(block,i,SectorBuffer);
        memcpy(ClusterBuffer[i], SectorBuffer, BytesPerSector);
    }
}
memcpy(SectorBuffer,ClusterBuffer[page],BytesPerSector);           (1)
Erase_Cluster(block); //写入之前，擦除当前簇；
for(i=0;i<SectorsPerCluster;i++) //一扇区接一扇区写入           (2)
{
    for(j=0;j<255;j++); //延时
    WritePage(block,i,ClusterBuffer[i]);
}
}

```

528

更新当前在 SectorBuffer 中扇区数据所在的簇，更新 SectorBuffer 中的数据 [4.6(1)]，其他数据保持不变[4.6(2)]。

程序清单4.7 扇区查找

```

DWORD SectorSearch(DWORD Sector, const char dirname[11], _FILE *file)
{
    BYTE* Cache;
    DIRENTRY *dir;
    unsigned int j;
    Cache = GetSectorData(Sector);           (1)
    if(Cache == NULL)
        return 0xffffffff;
    dir = (DIRENTRY *)Cache;
    for(j=0; (dir->deName[0] != '\0') && (j< BytesPerSector /
sizeof(DIRENTRY)); j++){           (2)
        if(memcmp((const char*)dir->deName, dirname, 11) == 0 &&
(! (dir->deAttributes & ATTR_VOLUME))) //如果目录名/文件名相同，且不是卷标项目
        {
            if(file != NULL)
            {
                memset((BYTE *)file, 0, sizeof(_FILE));           (3)
                file->DirSectorNum = Sector;
                file->DirIndex = j;
                memcpy((BYTE *)(&file->dir), (BYTE *)dir, sizeof(DIRENTRY));
            }
            return ClusterNum2SectorNum(dir->deStartCluster);       (4)
        }
    }
    dir++;
}
}

```

```
return 0xffffffff;
}
```

在所给的 sector 内寻找与 dirname 名称相同的项目。调用函数 GetSectorData() 将扇区 sector 的数据读入到内存的扇区缓冲区 SectorBuffer 中[4.7(1)], 然后以 32Bytes (DIRENTRY 的大小) 为单位, 逐个比较 DIRENTRY 的 dir->deName 成员, 直到找到与参数 dirname[11] 相同[4.7(2)], 将 DIRENTRY 的位置等信息存入参数 file 中[4.7(3)], 得到 DIRENTRY 的成员 deStartCluster (文件的起始簇号), 通过函数 ClusterNum2SectorNum() 转化为簇的起始块号, 返回[4.7(4)]。该函数的调用频率较高, 在定位文件时反复调用, 逐层目录寻找, 直到找到最底层目录, 返回其起始块号。

程序清单4.8 在整个文件内查找

```
DWORD fat_search(DWORD Sector, const char dirname[11], _FILE *file)
{
    unsigned int i;
    WORD Cluster;
    DWORD FirstSectorOfFile;
    if(Sector == FirstDirSector)//指定根目录
    {
        for(i=0; i<(RootDirCount * sizeof(DIRENTRY) / BytesPerSector); i++)//
在整个根目录区寻找
        {
            FirstSectorOfFile = SectorSearch(Sector++, dirname, file);      (1)
            if(FirstSectorOfFile != 0xffffffff)
                return FirstSectorOfFile; //返回目录/文件的第一簇
        }
    }
    else
    {
        Cluster = SectorNum2ClusterNum(Sector); //非根目录
        while(Cluster != 0xffff) //不是最后一簇
        {
            for(i=0; i< SectorsPerCluster; i++)
            {
                FirstSectorOfFile = SectorSearch(Sector++, dirname, file);      (1)
                if(FirstSectorOfFile != 0xffffffff)
                    return FirstSectorOfFile;
            }
            Cluster = GetNextClusterNum(Cluster);                          (2)
            Sector = ClusterNum2SectorNum(Cluster);
        }
    }
    return 0xffffffff; //找不到
}
```

528

在所给的 sector 开始寻找与 dirname 名称相同的项目, 当 sector 所在的簇还没有寻找完则继续寻找该簇的下一个 sector[4.8(1)], 当寻找到簇最后一个 sector, 则寻

找目录的下一个簇[4.8(2)], 直到找到同名的 DIRENTRY 或到达目录的最后一个簇为止。对扇区的寻找通过调用函数 SectorSearch() 实现[4.8(1)]。

程序清单4.9 将文件路径变为固定格式

```
char* get_valid_format(const char *fullpath)
{
    static char static_path[512];
    char* p=static_path;
    char path[80];
    char* ppath = path;
    int dir_len_count; //count dir len.
    int i;
    if(fullpath == NULL || strlen(fullpath) >=80 || *fullpath != '\\') (1)
    // 第一个字符是 '\
    return NULL;
    if(strlen(fullpath) > 1 && fullpath[strlen(fullpath)-1] == '\\') (1)
    //最后一个字符是 '\
        return NULL;
    strcpy(path, fullpath);
    memset(p, 0, 512);
    for(;;)
    {
        switch(*ppath)
        {
            case 0x00://结束符
            {
                if(dir_len_count != 0) // prev is not '\\'
                {
                    for(i=0; i<(11 - dir_len_count); i++)//未满足11字节, 用blank填满 (2)
                    {
                        *p = 0x20;
                        p++;
                    }
                }
                return static_path;//返回整个字符串
            }
            case '\\':
            {
                if(p != static_path) // this is not the first '\\'
                {
                    if(dir_len_count == 0)// more than one '\\'
                        return NULL;

                    for(i=0; i<(11 - dir_len_count); i++)
                    {
                        *p = 0x20;
                        p++;
                    }
                }
            }
        }
        ppath++;
    }
}
```

```
    }
}
*p = '\\';
p++;
ppath++;
dir_len_count = 0;
continue;
}
break;
case '.': (3)
{
    if(dir_len_count > 8 || dir_len_count == 0) // '\\.' check
        return NULL;
    if(ppath[1] == '.' || ppath[1] == '\\')
        return NULL;
    for(i=0; i<(8 - dir_len_count); i++)
    {
        *p = 0x20;
        p++;
    }
    dir_len_count = 8;
    ppath++;
    continue;
}
break;
case 0x22://非法字符
case 0x2A:
case 0x2B:
case 0x2C:
case 0x2F:
case 0x3A:
case 0x3B:
case 0x3C:
case 0x3D:
case 0x3E:
case 0x3F:
case 0x5B:
case 0x5D:
case 0x7C:
    return NULL;
default:
{
    if(*ppath < 0x20)
        return NULL;
}
break;
}
```

```

    *p = *ppath;
    dir_len_count ++;
    if(dir_len_count > 11)
        return NULL;
    p++;
    ppath++;
}
return static_path;
}

```

如果路径的格式不对则返回空[4.8(1)]，参数 fullpath 为文件的完整路径，为了在函数 SectorSearch() 中查找的方便，此处将文件路径变为固定的格式：每一级目录变为 11 位，不足 11 位的用空格来填充[4.8(2)]。如 dir1\directory2\name 经该函数变为 dir^^^^^^\directory2^\name^^^^^^（^代表空格）。对于 8.3 格式的文件名如 name.txt 经该函数变为 name^^^^txt，文件名占 8 位，扩展名占 3 位[4.8(3)]。其它格式为不合法路径，返回 NULL。

程序清单4.10 定位文件

```

DWORD fat_locate(const char *path, _FILE *file)
{
    DWORD Sector = FirstDirSector;
    char *p;
    p = get_valid_format(path); //判断是否合法 (1)
    if(p == NULL)
        return 0xffffffff;
    p++; //忽略第一个 '\'
    for(;;)
    {
        if(*p == NULL)
            return Sector; (4)
        Sector = fat_search(Sector, p, file); (3)
        if(Sector == 0xffffffff)
            return 0xffffffff;
        p+=12; (2)
    }
    return 0xffffffff;
}

```

定位文件，即在根目录区查找到它的第一级目录，在下一个簇查找到它的第二级目录……，直到最终定位文件。Path 为文件的完整路径，经过 get_valid_format() 函数变为固定格式的路径名[4.10(1)]，将路径按级分开，每一级的文件名长为 11Bytes[4.10(2)]，调用 fat_search() 实现分级查找，fat_search() 返回的值为父路径的起始簇的第一个扇区号[4.10(3)]，从而可以实现一级一级的查找，直到找到最终定位文件，返回最终定位文件起始簇的第一个扇区号[4.10(4)]。

程序清单4.11 在父目录登记文件

```

int AllocDir(DWORD ParentDirSectorNum, DIRENTRY* new_dir, _FILE * fp)

```

```

{
    BYTE* Cache;
    DIRENTRY *dir;
    DWORD i;
    WORD PrevCluster;
    WORD Cluster;
    DWORD DirSectorNum = ParentDirSectorNum;
    if(ParentDirSectorNum == FirstDirSector) //在根目录区 (1)
    {
        for(i=0; i<RootDirCount * sizeof(DIRENTRY) / BytesPerSector; i++)
        {
            Cache = GetSectorData(DirSectorNum);
            if(Cache == NULL)
                return 2;
            for(dir = (DIRENTRY *)Cache; (BYTE*)dir < Cache + BytesPerSector;
dir++) (2)
            {
                if(dir->deName[0] == '\0' || dir->deName[0] == 0xe5) (3)
                {
                    memcpy((BYTE *)dir, (BYTE *)new_dir, sizeof(DIRENTRY)); (4)
                    Flush(); (7)
                    if(fp) (5)
                    {
                        fp->DirSectorNum = DirSectorNum; //当前目录项所在的扇区地址
                        fp->DirIndex = ((BYTE*)dir - Cache) / sizeof(DIRENTRY); //目录索引，即该扇区中第x个目录项
                        memcpy((BYTE *)(&fp->dir), (BYTE *)new_dir, sizeof(DIRENTRY));
                    }
                    return 0; //操作成功，退出
                }
            }
            DirSectorNum++; //下一个根目录扇区
        }
        return 3;
    }
    else //不在根目录区
    {
        Cluster = SectorNum2ClusterNum(DirSectorNum); //
        while(Cluster != 0xffff)
        {
            for(i=0; i< SectorsPerCluster; i++) //在整个簇中搜索
            {
                Cache = GetSectorData(DirSectorNum);
                if(Cache == NULL)
                    return 2;
                for(dir = (DIRENTRY *)Cache; (BYTE*)dir < Cache + BytesPerSector;

```



```

dir++) (2)
{
    if(dir->deName[0] == '\\0' || dir->deName[0] == 0xe5 ||
dir->deName[0] == 0xff) (3)
    {
        memcpy((BYTE *)dir, (BYTE *)new_dir, sizeof(DIRENTRY)); (4)
        Flush(); (7)
        if(fp) (5)
        {
            fp->DirSectorNum = DirSectorNum;
            fp->DirIndex = ((BYTE*)dir - Cache) / sizeof(DIRENTRY);
            memcpy((BYTE *)(&fp->dir), (BYTE *)new_dir, sizeof(DIRENTRY));
        }
        return 0;
    }
    DirSectorNum++; //下一个扇区
}
PrevCluster = Cluster; //从FAT表中找出下一簇 (6)
Cluster = GetNextClusterNum(Cluster);
DirSectorNum = ClusterNum2SectorNum(Cluster); //刷新目录扇区
}
Cluster = AllocCluster(PrevCluster); //利用FAT表获得一个空簇 (8)
if(Cluster == 0xffff) //没有空簇
    return 4;
DirSectorNum = ClusterNum2SectorNum(Cluster); //空簇做新的目录扇区地址
Cache = GetSectorData(DirSectorNum);
if(Cache == NULL)
    return 2;
dir = (DIRENTRY *)Cache;
memcpy((BYTE *)dir, (BYTE *)new_dir, sizeof(DIRENTRY));
Flush();
if(fp)
{
    fp->DirSectorNum = DirSectorNum;
    fp->DirIndex = ((BYTE*)dir - Cache) / sizeof(DIRENTRY);
    memcpy((BYTE *)(&fp->dir), (BYTE *)new_dir, sizeof(DIRENTRY));
}
return 0;
}
return 5;
}

```

分为在根目录区登记文件和在非根目录区登记文件两种情况，在根目录区登记文件要从根目录的起始扇区到 0 簇的最后一个扇区，而在非根目录区要到目录文件的最后一个簇的末尾，实现的过程不同故分两种情况处理[4.11(1)]。

参数 ParentDirSectorNum 为父目录的起始扇区号，new_dir 传递要写入的 DIRENTRY 数据结构的值，同时为 fp 的 dir 变量赋值，fp 的 DirSectorNum（写入的扇区数）、DirIndex（在该扇区的偏移）变量定位写入的 DIRENTRY 结构的位置，可以在后续操作中方便的对 DIRENTRY 结构进行修改，写完后要及时更新 NAND Flash 的值（使用 Flush() 函数）[4.11(7)]。

从父目录区中写入一个文件或目录的 DIRENTRY 结构，以 32Bytes 为一个单位[4.11(2)]，根据 DIRENTRY 中的 deName[0] 成员寻找空闲的空间。0x00 表示空间空闲，0xe5 表示原 DIRENTRY 结构已被删除，可以使用，要在目录空间查找未被使用或已经删除的空间[4.11(3)]，然后将参数 new_dir 的值写入该位置[4.11(4)]，并将其在父目录的位置等信息记录到参数 fp 中[4.11(5)]。对于在非根目录中查找的情况如果在当前簇没有空闲空间，还要查找下一簇，直到文件末尾[4.11(6)]。如果目录文件的空间被全部占用，则要在 FAT 表中申请新的空间[4.11(8)]。

程序清单4.12 删除目录

```
int DeleteDir(_FILE *file) (1)
{
    BYTE* Cache;
    DIRENTRY *dir;
    Cache = GetSectorData(file->DirSectorNum); //文件在目录区的扇区地址 (2)
    if(Cache == NULL)
        return 1;
    dir = (DIRENTRY *)Cache;
    dir += file->DirIndex; //目录的索引号
    dir->deName[0] = 0xe5; //做删除标记 (3)
    Flush();
    return 0;
}
```

file 中的 DirSectorNum（写入的扇区数）、DirIndex（在该扇区的偏移）属性记录了目录的 DIRENTRY 结构在其父目录中的位置[4.12(1)]，调用 GetSectorData（）函数读出该扇区的数据到内存扇区数据缓冲器[4.12(2)]。的值找到文件（目录）在父目录中的位置，将 fp.dir.deName[0] 的值写为 0xe5，表示该文件（目录）被删除[4.12(3)]。

程序清单4.13 其他函数

(1) Intfat_format(unsigned charMedia,unsignedchar izeofdisk_M,unsigned char FilesysType)

文件系统格式化，将文件的相关信息写入起始扇区。

(2) unsigned char fatInit(void)

该函数主要用于从已有的存储介质中获得文件系统信息，并将其存入全局变量中。

(3) void FlushFAT()

将更新后的 FatCache 中的数据写入 NAND Flash 的 FAT 所在的扇区。

(4) WORD AllocCluster(WORD PrevClusterNum)

参数 PrevClusterNum 为当前簇的簇号，该函数从 FatCache 中查找值为零（即未被使

用)的簇号(LastAllocClusterNum),并将其值赋给 FatCache[PrevClusterNum],表示下一簇的簇号为 LastAllocClusterNum ,FatCache[LastAllocClusterNum]赋值 0xffff,表明其为文件的最后一个簇。

(5) WORD GetNextClusterNum(WORD ClusterNum)

返回值为 FatCache[ClusterNum],即文件下一簇的簇号。

(6)void FreeCluster(WORD StartClusterNum)

StartClusterNum 一般为文件的起使簇号,此函数应当是用于依次释放 fat 链,将文件占用的簇释放。

(7) WORD SectorNum2ClusterNum(DWORD SectorNum)

计算扇区号为 SectorNum 所在的簇号。

(8) DWORD ClusterNum2SectorNum(WORD ClusterNum)

计算簇号 ClusterNum 起使扇区号。

4.4 文件系统的 API 函数

上一节中的函数完成了文件系统的基本功能,文件系统的 API 函数的实现是通过反复调用这些函数实现的,故对上述函数的理解是理解程序的根本。文件系统的 API 函数为应用程序提供了丰富文件系统的接口,使用户不需考虑文件系统实现的细节,主要功能包括创建目录、删除目录、创建文件、打开文件、读文件、写文件、关闭文件、删除文件、重命名文件(目录)、移动文件、复制文件,显示目录的文件和子目录。

(1) 创建目录

程序清单4.14 创建目录

```
int fat_mkdir( const char *dirname)//仅1级目录: 父目录\新目录
{
    DIRENTRY dir;
    DWORD SectorNum;
    char path[512];
    char name[11];
    char *p;
    _FILE file;
    BYTE* Cache;
    DIRENTRY *pdir;
    WORD NewCluster;
    p = get_valid_format(dirname);
    if(p == NULL)
        return 1;
    if(fat_locate(dirname, NULL) != 0xffffffff)
        return 4;
    strncpy(name, &p[strlen(p)-11], 11);
    strcpy(path, dirname);
    p = strrchr(path, '\\');
    if(p == path) // it is root dir.
        *(p+1) = '\\0';
```

```

else
    *p = '\\0';
SectorNum = fat_locate(path, NULL); //获取第一级目录指向第一个扇区的地址 (5)
if(SectorNum == 0xffffffff)
    return 2;
memset((BYTE *)&dir, 0, sizeof(dir));
memcpy((BYTE *)&dir.deName, name, 11);
dir.deAttributes = ATTR_DIRECTORY; //目录属性
if(AllocDir(SectorNum, &dir, &file) != 0) (6)
    return 3;
NewCluster = AllocCluster(0); (7)
if(NewCluster == 0xffff)
    return 4;
Cache = GetSectorData(file.DirSectorNum); //将新的目录项所在扇区读入 (8)
if(Cache == NULL)
    return 6;
pdir = (DIRENTRY *)Cache;
pdir += file.DirIndex;
pdir->deStartCluster = NewCluster; //指向新的一簇 (9)
Flush(); (10)
return 0;
}

```

创建目录。Dirname 为创建完整的目录名，get_valid_format() 将 dirname 变为固定的格式[4.14(1)]，如果文件路径不合法返回空[4.14(2)]。通过 p 指针的操作，将完整的路径名分为父路径名和要新建的目录名[4.14(4)]，fat_locate() 返回父目录的第一个扇区号[4.14(5)]，AllocDir() 将新建目录的 DIRENTRY 结构写入父目录中，并通过参数 file 记下其在父目录中的位置[4.14(6)]。AllocCluster(0) 为新建目录分配一个簇[4.14(7)]，赋值给新建目录的 DIRENTRY 结构的 deStartCluster 成员[4.14(9)]，通过函数 GetSectorData() 读取目录的 DIRENTRY 结构[4.14(8)]，并将更新后的数据写入。使用 Flush() 函数将修改后的信息写入 NAND Flash 中[4.14(10)]。

创建新目录的限制：父目录必须已经创建，父目录中不存在与新建目录名相同的目录成员[4.14(3)]，父目录有足够的空间来写入新建目录的 DIRENTRY 结构，路径的格式要合法。

（2）删除目录

程序清单4.15 删除目录

```

int fat_rmdir( const char *dirname)
{
    DWORD SectorNum;
    _FILE file;
    char filename[13];
    if(fat_getfirst(dirname, filename) == 0) (1)
        return 3;
    //locate
}

```

```

SectorNum = fat_locate(dirname, &file); (2)
if(SectorNum == 0xffffffff) (6)
    return 4;
// is it a dir ?
if(!(file.dir.deAttributes & ATTR_DIRECTORY)) (3)
    return 6;
if(DeleteDir(&file) != 0) (4)
    return 5;
FreeCluster(file.dir.deStartCluster); (5)
return 0;
}

```

删除目录。dirname 为要删除的目录的完整的路径名。调用函数 fat_locate(), 参数 file 记录了目录的 DIRENTRY 结构在父目录的扇区号及在号内的偏移[4.15(2)], 调用 DeleteDir() 将 DIRENTRY 结构成员 deName[0] 设为 0xe5[4.15(4)], 表示该目录已经删除, 最后删除该目录所占用的簇空间[4.15(5)]。

删除目录的限制: 目录中不能有未删除的文件或子目录[4.15(1)], 属性必须为目录[4.15(3)], 路径的格式要合法[4.15(6)]。

(3) 创建文件

程序清单4.16 创建文件

```

int fat_creat(const char* filename, BYTE attribute)
{
    DIRENTRY dir;
    char path[512];
    char name[11];
    char *p;
    DWORD ParentDirSectorNum;
    _FILE file;
    DIRENTRY *pdir;
    WORD NewCluster;
    BYTE* Cache;
    p = get_valid_format(filename); (1)
    if(p == NULL) (2)
        return -2;
    if(fat_locate(filename, NULL) != 0xffffffff) (3)
        return -3;
    strncpy(name, &p[strlen(p)-11], 11); (4)
    strcpy(path, filename);
    p = strrchr(path, '\\');
    if(p==path)//add
        *(p+1)='\\0';//add
    else//add
        *p = '\\0';
    ParentDirSectorNum = fat_locate(path, NULL); (5)
    if(ParentDirSectorNum == 0xffffffff)
        return -4;
}

```

```

memset((BYTE *)&dir, 0, sizeof(dir));
memcpy((BYTE *) (dir.deName), name, 11);
dir.deAttributes = attribute;
dir.deFileSize = 0;
if(AllocDir(ParentDirSectorNum, &dir, &file) != 0) (9)
    return -5;
NewCluster = AllocCluster(0); (6)
if(NewCluster == 0xffff)
    return -6;
Cache = GetSectorData(file.DirSectorNum);
if(Cache == NULL)
    return -7;
pdir = (DIRENTRY *)Cache;
pdir += file.DirIndex;
pdir->deStartCluster= NewCluster; (7)
Flush(); (8)
return 0;
}

```

创建文件。与创建目录的过程比较相似，filename 为文件的完整路径，get_valid_format() 函数将其变为固定格式[4.16(1)]，通过对指针 p 的操作将完整路径分为父目录的路径和文件名[4.16(4)]，调用 fat_locate() 函数查找父目录的第一个扇区号[4.16(5)]，调用 AllocDir() 函数在父目录中找到空闲的空间并将文件的 DIRENTRY 结构写入该空间[4.16(5)]，参数 file 中的属性记录了空闲空间的位置。为文件分配新的簇[4.16(6)]，将该值赋值给 DIRENTRY 的 deStartCluster 属性[4.16(7)]，调用 Flush() 将更新后的信息写入 NAND Flash[4.16(8)]。

创建文件的限制：父目录已经创建[4.16(5)]，父目录中不存在同名的文件[4.16(9)]。

（4）打开文件

程序清单4.17 打开文件

```

_FILE handles[16]; (1)
int fat_open(const char* filename)
{
    int i;
    _FILE * fp = NULL;
    DWORD FirstSectorNum;
    for(i=0; i<16; i++) (2)
    {
        if(!handles[i].valid)
        {
            fp = &handles[i];
            break;
        }
    }
    if(fp == NULL)
        return -1;
}

```

```

FirstSectorNum = fat_locate(filename, fp);                                (3)
if(FirstSectorNum == 0xffffffff)
    return -2;
fp->StartSectorNum = FirstSectorNum;                                    (4)
fp->CurrentSectorNum = fp->StartSectorNum;                              (5)
fp->SectorOffset = 0;                                                  (6)
fp->offset = 0;                                                         (7)
fp->valid = 1;                                                         (8)
return i;                                                             (9)
}

```

打开文件。对文件进行读写等操作之前，应先将文件打开，handles 是一个 FILE 类型的全局变量数组，共 16 个元素表示最多同时打开 16 个文件[4.17(1)]。打开的过程就是在这个数组中查找一个 valid 属性为 0（表示未被使用）的数组成员[4.17(2)]，然后将 valid 属性置为 1（已使用）[4.17(7)]。filename 为文件的完整路径，调用 fat_locate() 找到文件起始块号，参数 fp 记录了文件的 DIRENTRY 在父目录中的位置[4.17(3)]。fp 的 StartSectorNum 表示文件的起始扇区号[4.17(4)]，CurrentSectorNum 表示当前扇区号[4.17(5)]，SectorOffset 表示当前读写指针偏移的扇区个数[4.17(6)]，offset 表示当前读写指针偏移的字节数[4.17(7)]。返回值 i 表示打开文件的文件号，文件的读写等操作要使用文件号做参数[4.17(9)]。

（5）读文件

读文件函数

```

unsigned int fat_read(int handle, void* buffer, unsigned int bytes)

```

handle 为当前打开的文件在 handles[16] 数组中号。Buffer 为字符指针，bytes 为要读出字节数。

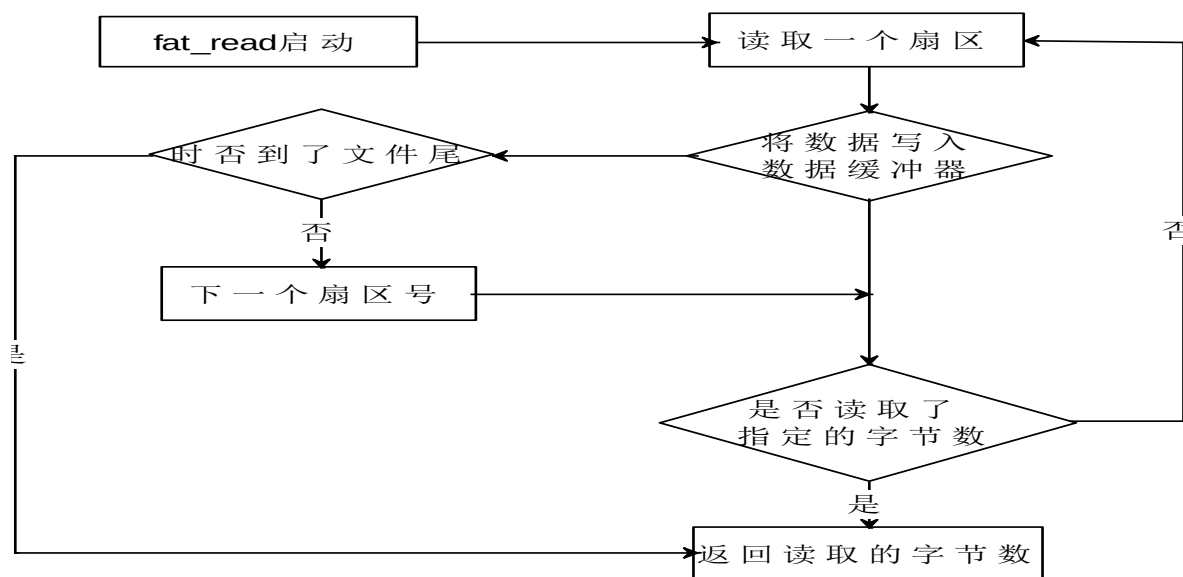


图 4.3 文件读流程图

（6）写文件

写文件函数

```
unsigned int fat_write(int handle, const char* buffer, unsigned int bytes)
```

handle 为当前打开的文件在 handles[16] 数组中号。Buffer 为字符指针，bytes 为要写入的字节数。

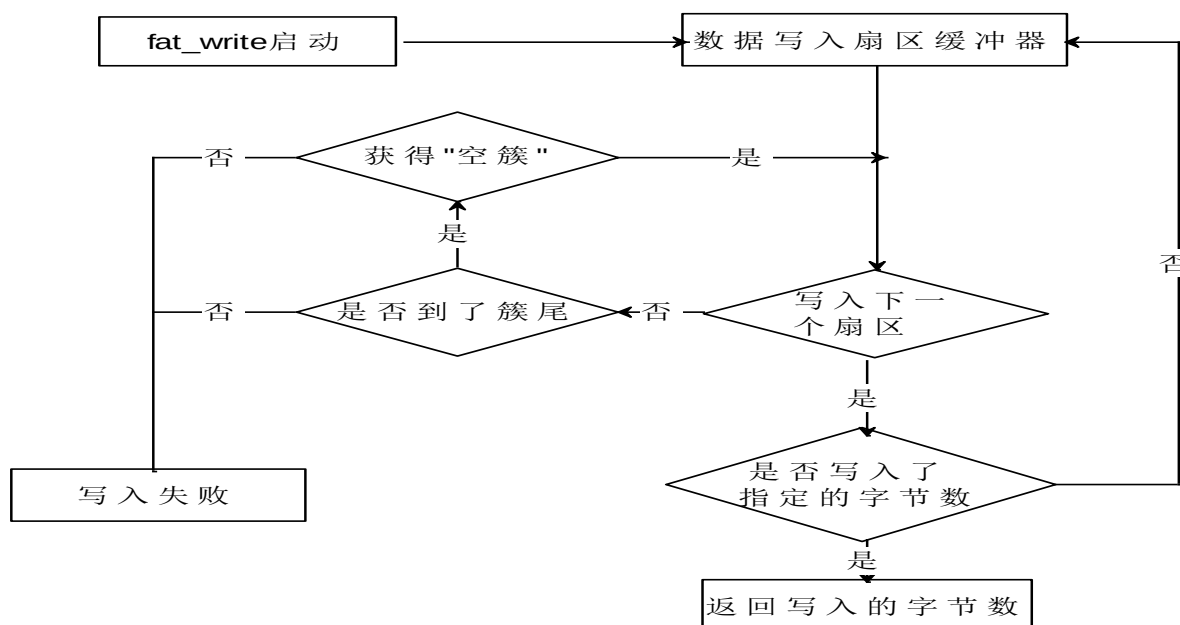


图 4.4 文件写流程

（7）关闭文件

程序清单4.18 关闭文件

```
int fat_close(int handle)
{
    _FILE *fp;
    BYTE* Cache;
    DIRENTRY *dir;
    if(handle < 0 || handle >= sizeof(handles)/sizeof(_FILE))
        return -1;
    fp = &handles[handle];
    Cache = GetSectorData(fp->DirSectorNum);
    if(Cache == NULL)
        return -2;
    dir = (DIRENTRY *)Cache;
    dir += fp->DirIndex;
    memcpy((BYTE *)dir, (BYTE *)&fp->dir, sizeof(DIRENTRY));
    Flush();
    FlushFAT();
    handles[handle].valid = 0;
    return 0;
}
```


handle 为当前打开的文件在 handles 数组中下标。handles 数组成员中的保存着文件 DIRENTRY 在父目录中的位置，将 DIRENTRY 数据结构的内容其读入到扇区缓冲器[4.18(1)]，将更新后的 DIRENTRY 数据结构的内容写入到相应位置[4.18(2)]，将更新后的 DIRENTRY 数据结构和 FAT 表写入到 NAND Flash 的相应位置[4.18(3)]。将 handles 数组中第 handle 个成员的 valid 属性设为 0（未被使用）[4.18(4)]。

（8）删除文件

程序清单4.19 删除文件

```
int fat_remove( const char *filename)
{
    DWORD SectorNum;
    _FILE file;
    SectorNum = fat_locate(filename, &file);                (1)
    if(SectorNum == 0xffffffff)                             (2)
        return 4;
    if(file.dir.deAttributes & ATTR_DIRECTORY)              (3)
        return 6;
    if(DeleteDir(&file) != 0)                                (4)
        return 5;
    FreeCluster(file.dir.deStartCluster);                    (5)
    return 0;
}
```

删除文件。filename 为文件的完整路径，调用函数 fat_locate()，参数 file 记录了文件的 DIRENTRY 在其父目录中位置[4.19(1)]，调用函数 DeleteDir() 函数将文件删除[4.19(4)]，调用 FreeCluster() 释放文件占用的簇[4.19(5)]。

删除文件的限制：路径正确[4.19(2)]，属性为文件[4.19(3)]。

（9）重命名文件

程序清单4.20 重命名文件

```
int fat_rename( const char *oldname, const char *newname )
{
    DIRENTRY dir;
    char path[512];
    char newpath[512];
    char name[11];
    char new_name[11];
    char *p;
    DWORD ParentDirSectorNum;
    _FILE old_file;
    p = get_valid_format(oldname);                            (1)
    if(p == NULL)
        return -2;
    if(fat_locate(oldname, &old_file) == 0xffffffff)         (3)
        return -3;
    strncpy(name, &p[strlen(p)-11], 11);                     (2)
    strcpy(path, oldname);
}
```

```

p = strrchr(path, '\\');
*p = '\\0';
p = get_valid_format(newpath); (4)
if(p == NULL) (5)
    return -2;
if(fat_locate(newpath, NULL) != 0xffffffff) (6)
    return -3;
strncpy(new_name, &p[strlen(p)-11], 11); (7)
ParentDirSectorNum = fat_locate(path, NULL); (8)
if(ParentDirSectorNum == 0xffffffff) (9)
    return -4;
memcpy((BYTE *)&dir, (BYTE *)&old_file.dir, sizeof(DIRENTRY)); (10)
memcpy((BYTE *)dir.deName, new_name, 11); (11)
if(AllocDir(ParentDirSectorNum, &dir, NULL) != 0) (12)
    return -5;
if(DeleteDir(&old_file) != 0) (13)
    return -6;
return 0;
}

```

oldname 为原文件（目录）的完整路径，newname 为（目录）修改后的文件路径。调用函数 fat_locate()，参数 file 记录了原文件的 DIRENTRY 在其父目录中位置，将其值取出赋给一个 DIRENTRY 类型的变量 dir。通过指针 p 将原文件路径分为父目录路径 path 和文件名 name[4.20(2)]，同理将 newname 分为原文件路径分为父目录路径和修改后文件名 nname[4.20(7)]，调用函数 fat_locate() 找到父目录的起始地址[4.20(8)]，将原文件的 DIRENTRY 数据结构复制给 DIRENTRY 类型的变量 dir[4.20(10)]，将新文件名赋值给 dir 的 deName 属性[4.20(11)]。调用函数 AllocDir()，将修改后的 DIRENTRY 结构写入了父目录中[4.20(12)]。调用 DeleteDir() 将原文件的 DIRENTRY 删除[4.20(13)]。

重命名文件（目录）的限制：oldname 和 newname 的格式要正确[4.20(1)] [4.19(4)]，，两个路径 oldname 和 newname 除文件名外其他路径要相同，原文件要存在[4.20(3)]，在该目录下不存在于新文件同名的文件[4.20(6)]。

（10）移动文件

程序清单4.21移动文件

```

int fat_move(char* from, char* to)
{
    DWORD FSectorNum;
    DWORD TSectorNum;
    _FILE file;
    FSectorNum=fat_locate(from, &file); (1)
    if(FSectorNum==0xffffffff) (2)
        return 4;
    TSectorNum=fat_locate(to, NULL); (3)
    if(TSectorNum==0xffffffff) (4)
        return 5;
}

```

```

    if(AllocDir(TSectorNum,&(file.dir),NULL) !=0)                (5)
        return 6;
    if(DeleteDir(&file) !=0)                                     (6)
        return 7;
    return 0;
}

```

from 为移动前文件的完整路径，to 为移动后文件所在的父目录的路径。调用函数 fat_locate() 得到移动前文件的起始扇区号，file 记录了文件的在父目录中位置且 file 的 dir 成员记录了原文件的 DIRENTRY 数据结构的内容[4.21(1)]，调用 fat_locate() 返回移动后文件所在的父目录的首扇区号 TSectorNum[4.21(3)]，AllocDir() 将原文件的 DIRENTRY 写入修改父目录所指向的簇[4.21(5)]。DeleteDir() 将文件在原父目录中的 DIRENTRY 删除[4.21(6)]。

(11) 复制文件

程序清单4.22 复制文件

```

int fat_copy(const char* from,const char* to)
{
    DWORD FSectorNum;
    DWORD TSectorNum;
    _FILE file;
    FSectorNum=fat_locate(from,&file);                            (1)
    if(FSectorNum==0xffffffff)                                    (2)
        return 4;
    TSectorNum=fat_locate(to,NULL);                              (3)
    if(TSectorNum==0xffffffff)                                    (4)
        return 5;
    if(AllocDir(TSectorNum,&(file.dir),NULL) !=0)                (5)
        return 6;
    return 0;
}

```

from 为复制前文件的完整路径，to 为复制后文件所在的父目录的路径。调用函数 fat_locate() 得到移动前文件的起始扇区号，file 记录了文件的在父目录中位置且 file 的 dir 成员记录了原文件的 DIRENTRY 数据结构的内容[4.22(1)]，调用 fat_locate() 返回移动后文件所在的父目录的首扇区号 TSectorNum[4.22(3)]，AllocDir() 将原文件的 DIRENTRY 写入修改父目录所指向的簇[4.22(5)]。

(12) 显示目录的文件和子目录

程序清单4.23 显示目录的文件和子目录

```

int fat_dir(const char* dirname)
{
    BYTE* Cache;
    DWORD Sector;
    WORD Cluster;
    DIRENTRY* dir;
    unsigned int i,j;
}

```

```

Sector=fat_locate(dirname, NULL); (1)
if (Sector==0xffffffff)
return 4;
while (Cluster!=0xffff) (2)
{
    for (i=0; i<SectorsPerCluster; i++)
    {
        Cache=GetSectorData (Sector++); (3)
        dir=(DIRENTRY*)Cache;
        for (j=0; j<BytesPerSector/sizeof (DIRENTRY); j++) (4)
        {
            if (dir->deName[0]!=0xff&& dir->deName[0]!=0xe5) (5)
            {
                if (dir->deAttributes==0x10) (6)
                    Uart_Printf("\t\t<DIR> \t\t");
                else
                    Uart_Printf("\t\t%ibytes\t\t", dir->deFileSize);
                Uart_Printf("%s\n", dir->deName);
            }
            dir++; (7)
        }
    }
    Cluster=GetNextClusterNum(Cluster); (8)
    Sector=ClusterNum2SectorNum(Cluster); (9)
    return 0;
}
}

```

dirname 为目录的完整路径，函数 fat_locate(dirname, NULL)返回目录的首个扇区地址[4.23(1)]，从该扇区开始逐个读取下一个扇区的数据[4.23(3)]，直到所在簇的最后一个扇区，如此循环直到文件的最后一个簇[4.23(2)]。未被使用的区间的值为 0xff，遍历目录的全部簇空间，以 DIRENTRY 的大小即 32Bytes 为单位[4.23(7)]，与 DIRENTRY 的起始 8 位即 deName[0]比较，将非 0xe5(已删除)，0xff(为空)的 DIRENTRY 的详细信息列出[4.23(5)]。如果属性为目录则显示<DIR>，属性为文件则显示文件的大小[4.23(6)]。

5 系统测试

在完成了该系统的全部开发工作后，对该系统进行了测试。下面给出测试的内容与结论，并根据测试的结果给出了软件改进的方案。

5.1 软件测试

对软件的测试主要是对指令执行结果的检测，检测结果如下：

(1) format 指令

格式化指令，指令格式为 format。

```
\>>format
Erase Nand-flash successfully!
Format Nand-flash Successfully!

format successful
OEM name      : -LiTai-
bytes per sector : 512
sectors per cluster : 32
reserved sectors: 1
RootDir Entry : 432
TotSectors    : 16384
FatSectors    : 2
Number of Fats : 2
First Fat Sector: 1
First Dir sector: 5
First Data Sect : 32
VolumeLabel   : ARMSYS44B0

\>>
```

图 5.1 格式化指令

输入 format 指令，文件系统进行格式化，显示的为文件系统的基本信息，如扇区大小、每簇的扇区数、根目录的目录项数目等。

(2) help 指令

帮助指令，显示指令菜单，格式为 help

```
\rootdir>>help
*****
*          UP-NetARM3000 Flash File System (FAT)          *
*          COM:115.2kbps,8Bit,NP,UART0                    *
*          cf(crfile)>>Creat a file                        *
*          md(mkdir)>>make a directory                     *
*          move>>Move file to another                     *
*          cd(chdir)>>Change directory                     *
*          copy>>Copy file form another                   *
*          del>>Delete a file                             *
*          dir>>Show content of directory                 *
*          rd(rmdir)>>Remove a directory                   *
*          ren(rename)>>Rename a directory                 *
*          rf(rdfile)>>Read file                           *
*          open>>Open a file                              *
*          wf(wtfile)>>Open a file                        *
*          format>>Format file system                    *
*          help>>List menu                                *
*          exit>>Exit file system                         *
*****
```

图 5.2 帮助指令

输入 help 指令，显示菜单：功能与其对应的指令。

(3) 目录操作相关指令

包括 md(mkdir) 指令、 rd(rmdir) 指令和 dir 指令。

创建文件夹指令 md(mkdir)，指令格式为 md (mkdir) dirname；删除文件夹指令 rd(rmdir)，指令格式为 rd (rmdir) dirname；显示目录中的子目录和文件指令 dir，指令格式为 dir。

注：在符号">>"左边的为当前目录，右边的为指令，其它的行为命令的运行结果或中间结果，图 5.4、图 5.5 中同样处理。

```
\>>md rootdir
make directory\rootdir
\>>cd rootdir
change dir:\rootdir
\rootdir>>md dir2
make directory\rootdir\dir2
\rootdir>>md dir1
make directory\rootdir\dir1
\rootdir>>mkdir dir3
make directory\rootdir\dir3
\rootdir>>dir
4-0-100      <DIR>      .      +
4-0-100      <DIR>      ..     +
4-0-0        <DIR>      dir2   +
4-0-0        <DIR>      dir1   +
4-0-100      <DIR>      dir3   +

show directory of \rootdir
\rootdir>>rd dir3
remove dir:\rootdir\dir3
\rootdir>>dir
4-0-100      <DIR>      .      +
4-0-100      <DIR>      ..     +
4-0-0        <DIR>      dir2   +
4-0-0        <DIR>      dir1   +

show directory of \rootdir
\rootdir>>
```

图 5.3 创建（删除）目录和显示目录内容指令

md(mkdir) 为创建目录指令，首先在根目录创建目录 rootdir。cd 指令更改当前目录，进入刚创建的 rootdir 目录，然后在 rootdir 目录内创建了三个目录 dir1、dir2 和 dir3。dir 指令显示目录 rootdir 中的内容，头两项分别为 rootdir 目录本身和它的父目录。rd 指令将 dir3 目录删除，重新显示 rootdir 中的内容，dir3 已经删除。

(4) 文件的创建、删除和读写指令

包括 cf(crfile) 指令、open 指令、wf(wtfile) 指令、rf(rdfile) 指令、close 指令、del 指令。

创建文件指令 cf(crfile)，指令格式为 cf(crfile) filename；打开文件指令 open，指令格式为 open filename；写文件指令 wf(wtfile)，指令格式为 wf(wtfile) filename；读文件指令 rf(rdfile)，指令格式为 rf(rdfile) filename；关闭文件指令 close，指令格式为 close；删除文件指令 del，指令格式为 del filename。

```

\rootdir>>cf file1
make file:\rootdir\file1
create a file
\rootdir>>cf file2
make file:\rootdir\file2
create a file
\rootdir>>open file1
File location:\rootdir\file1
open a file0
\rootdir>>wf
write a file:
Input content of file:

write file 0success16bytes
\rootdir>>rf
there are 16 bytes in file
read success
content of file1

\rootdir>>close
close file 0
\rootdir>>open file2
File location:\rootdir\file2
open a file0
\rootdir>>wf
write a file:
Input content of file:

write file 0success16bytes
\rootdir>>rf
there are 16 bytes in file
read success
content of file2

\rootdir>>close
close file 0
\rootdir>>open file2
File location:\rootdir\file2
open a file0
\rootdir>>wf
write a file:
Input content of file:

write file 0success16bytes
\rootdir>>rf
there are 16 bytes in file
read success
content of file2

\rootdir>>del file2
delete file:\rootdir\file2
\rootdir>>dir
4-0-100      <DIR>      .
4-0-100      <DIR>      ..
4-0-0        <DIR>      dir2
4-0-0        <DIR>      dir1
577-517-100      16bytes      file1
577-517-100      16bytes      file2

show directory of \rootdir
\rootdir>>del file2
delete file:\rootdir\file2
\rootdir>>dir
4-0-100      <DIR>      .
4-0-100      <DIR>      ..
4-0-0        <DIR>      dir2
4-0-0        <DIR>      dir1
577-517-100      16bytes      file1

show directory of \rootdir
\rootdir>>

```

图 5.4 文件的创建删除和读写指令

通过 cf(crfile) 指令分别在目录 rootdir 创建了文件 file1 和 file2, 分别调用相应指令打开、写、读、关闭两个文件。调用 del 指令删除文件 file2, 通过比较 dir 指令显示的内容, 可见 file2 被删除。

(5) 其它指令

文件移动指令 move, 指令格式为 move oldpath newpath; 文件复制指令 copy, 指令格式为 copy oldpath newpath; 重命名指令 ren(rename), 指令格式 ren(rename) oldname newname。改变路径指令 cd(chdir), 指令格式为 cd(chdir) newpath。

```

\rootdir>>rename file1 file3
rename form \rootdir\file1 to file3
rename success
\rootdir>>cf file4
make file:\rootdir\file4
create a file
\rootdir>>dir
4-0-100      <DIR>      .
4-0-100      <DIR>      ..
4-0-0        <DIR>      dir2
4-0-0        <DIR>      dir1
577-517-100      16bytes      file3
577-517-100      0bytes      file4

show directory of \rootdir
\rootdir>>move file4 rootdir\dir2
move file from \rootdir\file4 to \rootdir\dir2
\rootdir>>copy file3 rootdir\dir1
copy file from \rootdir\file3 to \rootdir\dir1
move file from \rootdir\file3 to \rootdir\dir1
\rootdir>>cd dir2

change dir:\rootdir\dir2
\rootdir\dir2>>dir
4-0-0        <DIR>      .
4-0-0        <DIR>      ..
577-517-100      0bytes      file4

show directory of \rootdir\dir2
\rootdir\dir2>>cd ..
change dir:
\rootdir>>cd dir1
change dir:\rootdir\dir1
\rootdir\dir1>>dir
4-0-0        <DIR>      .
4-0-0        <DIR>      ..
577-517-100      16bytes      file3

show directory of \rootdir\dir1
\rootdir\dir1>>

```

图 5.5 其它指令

接图 5.4 的运行结果，调用运行 `rename` 指令将文件 `file1` 重命名 `file3`，创建文件 `file4`，调用运行 `dir` 指令显示目录的子目录和文件，可见前两个指令运行正确。调用 `move` 指令将 `file4` 从 `rootdir` 移动到它的子目录 `dir2` 中，调用 `copy` 指令将 `file1` 复制到 `rootdir` 的子目录 `dir1` 中。调用指令 `cd` 分别跳转到目录 `dir2` 和 `dir1`，显示目录的内容，可见指令完成了文件的复制和移动功能。

(6) `exit` 指令

退出系统，指令格式 `exit`。

(7) 总结

上面验证了文件系统指令的功能，包括格式化指令 `format`、帮助指令 `help`、创建目录指令 `md(mkdir)`、删除目录指令 `rd(rmdir)`、显示目录的子目录和文件指令 `dir`、创建文件指令 `cf(crfile)`、打开文件指令 `open`、读文件指令 `rf(rdfile)`、写文件指令 `wf(wtfile)`、关闭文件指令 `close`、删除文件指令 `del`、重命名文件（目录）指令 `rename`、移动文件指令 `move`、复制文件指令 `copy`、退出系统指令 `exit`。基本完成了预定的功能。

5.2 改建思想

目前 FAT 总体设计中的要求的功能已经基本实现了，但是由于时间有限，文件系统存在很多不完善之处，主要有以下 4 方面需要改进。

(1) 日期的显示

对文件创建、修改、最后访问的日期等方面的功能没有实现，`dir` 指令显示一个目录的文件和子目录，第一个属性为文件的创建日期。

(2) 文件的读写指针的设置

每个打开的文件都有当前文件的读写指针，对文件的读写从当前读写指针的位置开始，此处省略了该部分能容，文件的读写指针都是从默认的 0 开始，即在写操作完成关闭文件之后，再进行写操作则从文件的 0 偏移开始。

(3) 没有使用操作系统

同一时刻只有一个用户操作，操作其它文件时要先将打开的文件关闭，然后再打开要操作的文件。

(4) FAT 表的改进

FAT 表是文件系统中的—个重要的结构，如 FAT 表受到破坏则整个文件系统就会崩溃，该系统使用了两个 FAT 表都位于第 0 簇，如果 0 簇的数据受到破坏，文件系统就不能恢复。可将 FAT 表备份数目增加，如可以占用 10 个簇，一个 FAT 表大小为 2k，10 个簇可有 80 个 FAT 表备份，每打开一次文件系统使用下一个 FAT 表，修改后的 FAT 表只写入该 FAT 表，如某个扇区或簇的数据受到破坏，则使用最近一次的 FAT 表，虽然会有所损失但不至于导致文件系统崩溃。

总结

紧张而充实的毕业设计即将结束了。回顾在这段时间的工作，感慨颇多。毕业设计是对我们每名学生智力、毅力的一次艰巨考验，是对我们每名学生知识体系的一次丰富升华，是对我们每名学生整个大学的学习成果的一次检验。在此之前对嵌入式的了解很少，在学习和设计的过程中翻阅了很多相关资料，阅读了很多相关的源代码，包括汇编语言和 C 语言，从而对嵌入式有了一个整体而深入的了解，在学习和设计中要注意同老师和同学的交流，大家对问题的理解各有所长，通过交流可以取长补短，大大减少了学习新知识的时间。善于利用网络上的资源很重要，在网上有很多有用的资料和类似的源代码，对于已有的资料要善于利用，充分的理解程序中的思想，然后加上自己的想法，“站在巨人的肩膀上才能看的更远”，对于没有的要敢于创新。

本次毕业设计完成了 FAT 格式的嵌入式文件系统，是利用网络上已有资源和自己创新的结合，在理解源代码的基础上，利用已有函数的功能增加三个文件系统 API 函数 `fat_dir`、`fat_move`、`fat_copy`，增加指令识别功能，实现 PC 和嵌入式平台的交互，向嵌入式平台发送指令，并检验指令执行的结果。同时，毕业设计只是完成了文件系统的基本功能，并没有深入考虑一些可能会遇到的意外情况，对 Flash 文件系统各种性能的研究与考虑很少，与实际的应用相差很远。这也正是以后要努力的方向。

致谢

经过几个月的努力准备、学习与尝试，现在毕业设计终于如期完成。从开始对嵌入式知识的一无所知，到现在能应用嵌入式系统完成一些基本的功能，这种进步的取得离不开陈智育老师和实验室所有同学的极大帮助。

陈老师在嵌入式方面有很深的造诣，在软件开发过程中的很多疑难问题陈老师总能给出完美的解答。在论文的撰写和格式的修改方面，陈老师严肃认真和一丝不苟的精神是我以后努力学习的方向。在与老师的交流过程中感受到了很多作为一个学者所应具有的优秀品质。

没有陈老师的帮助、指引，毕业设计不会这么顺利的完成，在此我向陈老师致以深深的谢意。

在毕业设计的过程中，王立宏老师、董浩老师、王建华老师在作品的验收和论文格式的建议方面付出了很大的心血，在此向这些老师表示深深的谢意。

参考文献

- [1] 王田苗. 嵌入式系统设计与实例开发. 北京: 清华大学出版社, 2004
- [2] 贾智平,张瑞华. 嵌入式系统原理与接口技术. 北京: 清华大学出版社, 2007
- [3] 汤子赢,哲风屏,汤小丹. 计算机操作系统. 西安: 电子科技大学出版社, 2004
- [4] 任满杰,刘树刚,李红军. 操作系统原理实用教程. 北京: 电子工业出版社, 2006
- [5] 郑玲,李为. 计算机专业毕业设计指导. 北京: 清华大学出版社, 2007
- [6] 邵贝贝. $\mu C/OS-II$ 一源码公开的实时嵌入式操作系统. 北京: 中国电力出版社, 2002
- [6] 谭浩强,张基温著.C 语言程序设计教程. 北京:高等教育出版社,2007